

2024 年第四届长三角高校数学建模竞赛

题目 基于回归分析和机器学习的分子物理化学性质的研究

摘要:

随着人工智能技术的发展,化学领域实现了从传统试错范式到智能化发展新范式的飞跃。基于大数据及机器模型的预测研究,对我国化学领域的创新具有重要意义。

针对问题一,本文首先对原始数据集进行**数据清洗**,对缺失值和异常值进行检验和处理,并对样本**连续数据**和**离散数据**进行了详尽的**探索性分析**,研究了各指标之间的相关性,为后续的数据分析提供数据基础。基于散点图初步判断 id 与 y_2 之间的关系,通过多项式回归、高斯函数回归和傅里叶函数回归等方法拟合两者之间的关系。**傅里叶函数回归**的拟合效果明显好于多项式回归和高斯函数回归,其 R^2 为 0.9988、SSE 为 0.1372、RMSE 为 0.00082。

针对问题二,本文为了减少样本数据的维度进行**特征指标选取**,提取与目标变量 y_1 显著相关的特征指标,采用了 **Spearman 相关系数法**和**随机森林特征选择法**,综合考虑两种方法,选取指标 x_4, x_6, x_7 。本文将选取的特征指标作为输入, y_1 作为输出,基于决策树、XGBoost、SVR 和 BPNN 等机器学习回归算法,构建分子化学性质预测模型。求解得到 **BPNN 预测模型**的预测效果明显高于其他模型,对测试集的预测准确率高达 **99%**,其 RMSE 为 0.41879、MAE 为 0.26825、 R^2 为 0.99978。

针对问题三,本文基于问题二的特征提取方式,选取特征指标 x_1, x_3 作为输入,目标变量 y_3 作为输出,通过多项式回归的方式拟合,可得完全二次型的拟合效果最好,显著性最强,通过**逐步回归**的方式,剔除**完全二次型**中的无关变量,最终得到预测效果最好的拟合函数依然为完全二次型。本文对 y_3 进行灵敏度分析,使得单个变量的值在范围内波动,观察 y_3 的变化趋势,可知 x_4 对 y_3 的影响更大,对其预测具有显著性影响。

针对问题四,在 class 为定类数据的情况下,本文为解决样本数据中的冗余特征变量和无用数据,采用了**互信息法**和**递归特征消除法**,综合考虑两种方法,选取了与 class 相关性较强的特征指标 y_1, x_1, x_4, x_6, x_7 , 本文将选取的特征指标变量作为输入, class 作为输出,基于决策树、XGBoost、SVR 和 BPNN 等机器学习分类算法。求解可得 **XGBoost 分类模型**的效果明显好于其他模型,对测试集的分类准确率高达 **93%**,其精度为 0.9345、召回率为 0.9142、F1 为 0.8957、AUC 为 0.9049。

针对问题五,本研究将综合问题二至四中的特征指标选取方式,确定不同情况下各目标变量的相应的特征指标。考虑到分类问题中样本类别的不平衡性会影响机器学习模型对少数类别的学习,进而导致分类准确率下降。本文选择对少数类别数据进行**SMOTE 过采样**,并对预测效果最好的 BPNN 模型和分类效果最好的 XGBoost 模型,分别采用粒子群(PSO)算法进行超参数寻优,重新进行训练。可得 **PSO-BPNN 预测模型**对测试集的预测准确率高达 **99.99%**。RMSE 为 0.29×10^{-5} MAE 为 0.00027、 R^2 为 0.9999。**PSO-XGBoost 分类模型**对测试集的分类准确率高达 **99.14%**,其精度为 0.96、召回率为 0.97、F1 为 0.96、AUC 为 0.97。

本文最后对模型的优缺点进行了评价,并且对模型进行了推广与应用。

关键词: 机器学习回归; 机器学习分类; XGBoost; BP 神经网络; PSO 算法

目录

一、问题背景与重述.....	1
1.1 问题背景.....	1
1.2 问题重述.....	1
二、问题分析.....	1
2.1 问题一的分析.....	1
2.2 问题二的分析.....	1
2.3 问题三的分析.....	2
2.4 问题四的分析.....	2
2.5 问题五的分析.....	2
三、模型假设.....	3
四、符号及变量说明.....	3
五、模型建立与求解.....	4
5.1 问题一模型建立与求解.....	4
5.1.1 数据预处理.....	4
(1)原始数据集概述.....	4
(2)数据清洗.....	4
5.1.2 数据探索性分析.....	5
5.1.3 直接通过 id 预测 y_2 的模型建立.....	7
5.1.4 直接通过 id 预测 y_2 的模型求解.....	8
5.2 问题二模型建立与求解.....	10
5.2.1 特征指标的选取.....	10
5.2.2 机器学习回归模型的建立.....	12
5.2.3 机器学习回归模型的求解.....	18
5.3 问题三模型的建立与求解.....	20
5.3.1 特征指标的选取.....	20
5.3.3 函数拟合模型的建立与求解.....	21
5.3.4 函数拟合精度误差分析.....	22
5.3.5 灵敏度分析.....	23
5.4 问题四模型的建立.....	24
5.4.1 特征指标选取.....	24
5.4.2 机器学习分类模型的建立.....	26
5.4.3 机器学习分类模型的求解.....	27
5.5 问题五模型的建立.....	30
5.5.1 PSO 算法原理.....	30
5.5.2 机器学习回归 PSO-BPNN 模型建立.....	31
5.5.3 机器学习分类 PSO-XGBoost 模型建立.....	32
六、模型的评价与改进.....	36
七、模型的推广与应用.....	37
参考文献.....	39

一、问题背景与重述

1.1 问题背景

物理和化学研究对象日益复杂化,传统研究范式依赖“穷举”、“试错”和“重复”手段,难以实现全局最优搜索。为此,中国科技大学开发了一个先进的机器化学家平台。该平台在大数据和智能模型的双驱动下,实现了化学合成、表征和测试的全流程自动化开发,超越了欧美同类装置,涵盖了光催化与电催化材料、发光分子、导电分子、磁学材料和光学薄膜材料等领域。

人工智能平台独特的计算大脑通过大数据挖掘、物理模型、量子化学、机器学习等方法,使智能模型更好地理解 and 创造化学。这一系统对化学科学产生重大影响,摆脱传统研究范式限制,展示了“最强化学大脑”指导的智能新范式优势,确立了我国在智能化学创新领域的全球领先地位。

基于数据的预测研究是平台的重要工作之一^[1]。附件 `data.csv` 提供了 20 万个化学分子的物理化学性质数据。`predict.csv` 包含 2580 个分子的性质数据,`submit.csv` 需要预测这些分子的 $y_1 \sim y_3$ 和“class”属性。本研究旨在通过数据分析与建模,帮助机器化学家完成预测任务。

1.2 问题重述

问题 1 的重述:对给定数据进行预处理,探索分子 `id` 与 y_2 之间的函数关系,并尝试通过 `id` 直接预测 y_2 。

问题 2 的重述:从 `data.csv` 中选择不超过 10 个特征指标,建立 y_1 预测模型。

问题 3 的重述:分析 y_3 与数据中的特征指标之间的函数关系,建立数学模型预测 y_3 ,确定影响较大的特征指标,并进行灵敏度分析。

问题 4 的重述:分析 `class` 与数据中的特征指标之间的关系,基于物理化学性质建立分子类别的预测模型,确定对分类结果影响较大的特征指标。

问题 5 的重述:探讨是否有更好的预测方法以提高模型的预测精度,详细描述可能的方法,并重新对 y_1 、 y_3 和 `class` 进行预测,论证新方法的优越性。

二、问题分析

2.1 问题一的分析

针对问题一,本文首先对样本数据进行数据预处理,包括缺失值和异常值的检测和处理。本文对连续数据和离散数据进行了详尽的探索性分析,并深入研究了指标之间的相关性。这样的分析能够更好地理解数据特征和相互关系,为后续的数据分析提供了有力的数据基础。基于 `id` 与 y_2 之间的散点图,本文直观地观察两者之间的大致关系,尝试应用多项式回归、高斯函数和傅里叶函数等拟合方法,以选择最优的拟合函数,基于最优的拟合函数,实现 `id` 对 y_2 进行准确预测。

2.2 问题二的分析

针对问题二,发现样本数据中存在过多的分子物理化学性质。为了减少维度和提取与 y_1 显著相关的特征,本文采用了 Spearman 相关系数法和随机森林特征选择法,反映变量之间的相关作用和依赖关系,确定与 y_1 相关性较强的特征指标。将选取的特征指标变量作为输入, y_1 作为输出,我们在决策树、XGBoost、SVR 和 BPNN 等常用机器学习模型中进行训练。通过比拟定相关评价指标,判断在训练过程中的模型预测效果,选出预测效果最好的模型,实现对于 y_1 的预测。

2.3 问题三的分析

针对问题三，本文基于问题二的特征提取方式，确定了与目标变量 y_3 显著相关的特征指标。使用多元非线性回归方法，以确定特征指标变量与目标变量 y_3 之间的拟合函数。通过逐步回归的方式，剔除相对来说无用的变量，最终得到预测效果最佳的拟合函数。同时，本文对目标变量 y_3 进行灵敏度分析，通过人为使特征变量波动，观察 y_3 的变化程度，以评估不同特征变量对目标变量的影响重要性和贡献程度。

2.4 问题四的分析

针对问题四，为解决样本数据中存在过多的分子物理化学性质，且目标变量为定类数据，本文选择采用互信息法和递归特征消除法，选取与目标变量 $class$ 相关性较强的特征指标。将选取的特征指标变量作为输入， y_1 作为输出，我们在决策树、XGBoost、SVR 和 BPNN 等常用机器学习分类模型中进行训练。通过机器学习分类模型的精度、召回率、准确度、F1 和 AUC 等评价指标，判断在训练过程中的模型分类效果，选出分类效果最好的模型，实现对于 $class$ 的分类预测。

2.5 问题五的分析

针对问题五，本研究将根据问题二至四中的特征指标选取方式，确定相应的特征指标。采用粒子群优化(PSO)算法对训练效果最好的模型进行超参数寻优。考虑到分类问题中样本类别的不平衡性可能会影响机器学习模型对少数类别的学习，进而导致分类准确率下降。因此，本文选择对少数类别数据进行 SMOTE 过采样，以重新进行机器学习训练。最终给出改进后最优的机器学习回归算法以及机器学习分类算法。

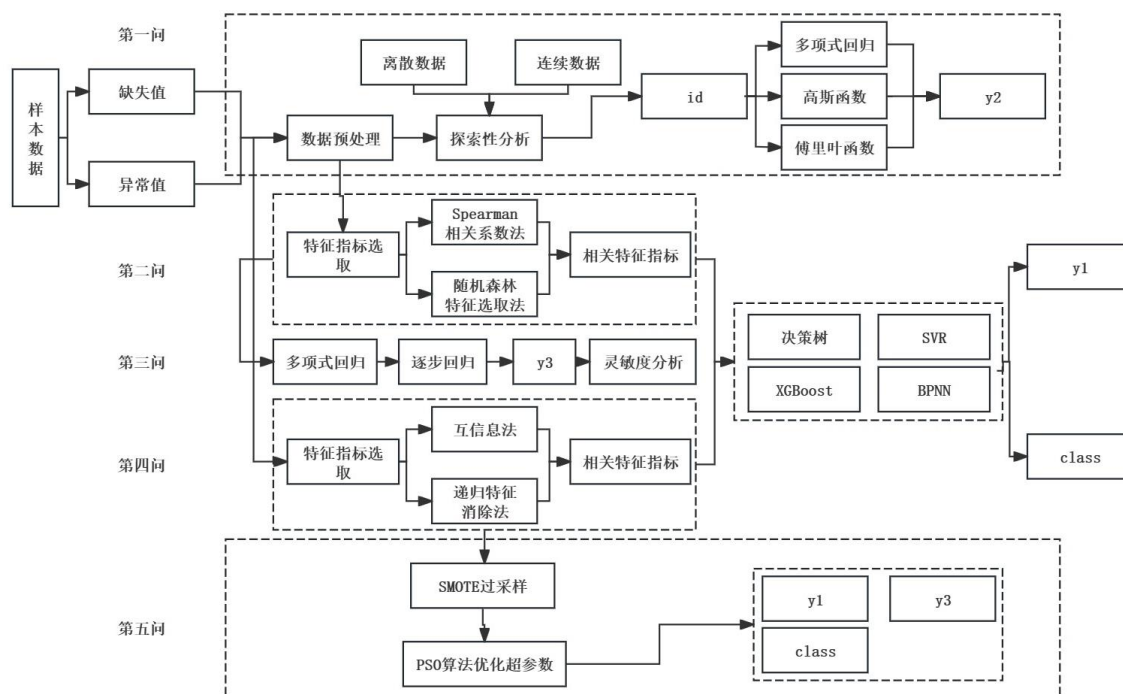


图 1 问题分析流程图

三、模型假设

- 1.假设题目中所给的数据真实、有效。
- 2.假设模型在不同的数据集上表现稳定，即模型的预测能力不会因数据集的变化而显著改变。
- 3.假设假设所选择的特征指标之间是相互独立的，即它们之间没有显著的相关性。这有助于简化模型，避免多重共线性问题。

四、符号及变量说明

符号及变量	说明	单位
x_{id}	分子序号	/
x_i	分子性质指标($i = 1, 2, \dots, 100$)	/
y_j	分子性质指标($j = 1, 2, 3$)	/
R^2	数据拟合优度	/
MSE	均方误差	/
$RMSE$	均方根误差	/
a	拟合的估计参数 1	/
b	拟合的估计参数 2	/
ρ_s	Spearman 相关系数	/
GI	平均基尼不纯度	/
MDG	平均值基尼不纯度下降	/
ω	权向量	/
η	拉格朗日乘子	/
$\bar{y}_i$	模型实测值的均值	/
y_i^*	模型预测值	/
AC	模型预测的准确度	/
P	模型预测的精确度	/
R	模型预测的召回率	/
$F1$	精确度和召回率的调和平均	/
TP	真正例	/
TR	真负例	/
FP	假正例	/
FN	假负例	/

五、模型建立与求解

5.1 问题一模型建立与求解

5.1.1 数据预处理

数据预处理是在机器学习和数据分析中对原始数据进行清洗、转换和处理的过程，其主要目的是提高数据质量、减少噪声和异常值的影响、使数据适合模型训练，从而提高模型的性能和泛化能力。我们将从数据集概述、数据清洗两部分对数据进行预处理。

(1)原始数据集概述

对原始数据集进行概述，data.csv 数据集提供了 20 万个化学分子的物理化学性质的数据，给出数据集概述表(部分)和数据的描述统计量，见表 1：

表 1 数据集概述表（部分）及数据的描述统计量

特征名称	数据类型	平均值 $\bar{x}$	中位数 M	标准差 σ	最大值 max	最小值 min
id	序号	/	/	/	20000	1
class	离散变量	/	/	/	4	1
y_1	连续变量	-492.669	-462.055	125.410	-347.260	-1725.10
y_2	连续变量	-0.231	-0.234	0.024	-0.153	-0.331
y_3	连续变量	0.006	0.010	0.045	0.101	-0.220
x_1	连续变量	0.188	0.187	0.037	0.359	0.055
x_2	连续变量	0.000063	0.000062	0.000007	0.000105	0.000036
x_3	连续变量	0.237	0.240	0.054	0.407	0.036
x_4	连续变量	-492.622	-462.010	125.409	-347.221	-1725.06
...	...	...	...	...	...	...
x_{98}	连续变量	20614.769	20631.912	11907.164	41193.415	0.034
x_{99}	连续变量	12255.690	12271.990	7081.966	24469.070	0.015
x_{100}	连续变量	364.170	363.842	210.555	728.991	0.001

通过表 1，可以看出该数据集的大小为 20000×105。id 是序号变量，用于区分不同类型分子的数据。class 是离散变量，用于区分不同分子的类别，经过统计分析可知 class 共有 4 类。 $y_1 \sim y_3$ 和 $x_1 \sim x_{100}$ 是分子的 103 个物理化学性质。同时可以看出相关指标的基本特征，为后续数据处理做准备。

(2)数据清洗

数据清洗确保数据的质量、准确性和一致性，为后续分析和建模提供可靠的基础。通过清洗数据，可以识别缺失值和异常值，提高数据的和可用性。

对数据进行缺失值识别，发现该数据集并未存在缺失值。

对数据进行异常值处理：考虑绘制 $y_1 \sim y_3$ 和 $x_1 \sim x_{100}$ 分子的 103 个物理化学性质的箱线图，通过最小值、下四分位数 (Q1)、中位数 (Q2)、上四分位数 (Q3) 和最大值。对数据的异常值进行识别与分析，进一步对数据进行处理。

绘制部分分子性质指标的箱线图，见图 2：

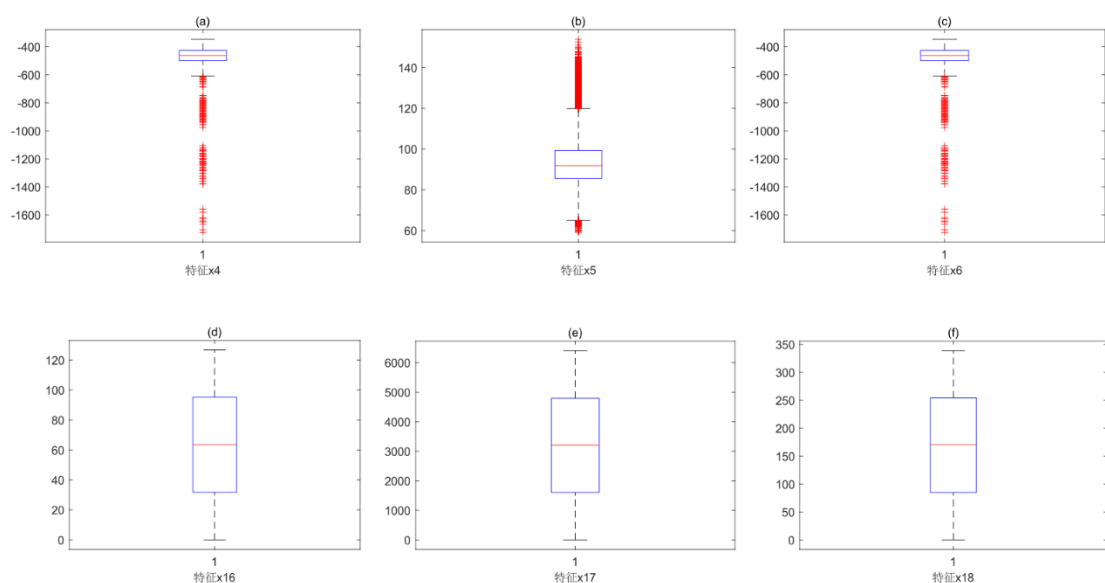


图 2 分子性质的箱线图（部分）

图 2 直观清晰的展示了分子性质指标 $x_i (i = 4, 5, 6, 16, 17, 18)$ 的箱线图，通过分析可知分子指标 $x_i (i = 4, 5, 6)$ 存在部分离群点，根据文献^[2]可知，化学分子物质中的任何一个“微小的”差异都可能导致其性质的明显差异。例如：同种元素的同位素之间尽管只是中子数不一样，但对物质结构和性质的影响却不容忽视。所以对于上述箱线图中存在离群点的分子性质指标不进行剔除。相反，认为这些离群点可能包含了影响分子性质的有价值的信息，代表了一些特殊情况或者异常现象，这些现象对于我们理解分子结构和性质之间的关系具有重要意义。

5.1.2 数据探索性分析

数据探索性分析的意义在于初步通过对数据的细致观察和分析，通过可视化的方式直观清晰的发现数据中的模式、趋势，指导后续特征选择和建模。

(1) 离散变量的探索性分析

考虑对四分类离散变量 class 进行不同类的数量统计，统计直方图见图 3：

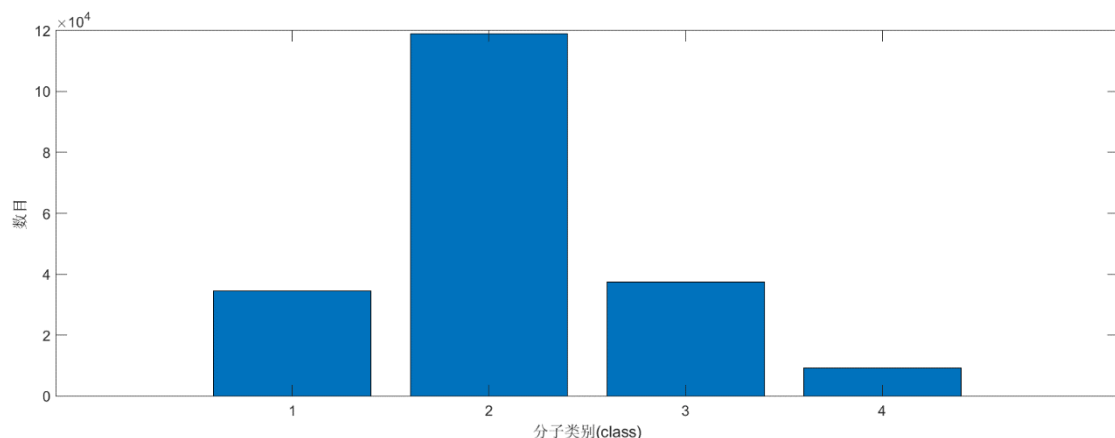


图 3 分子类别 class 的不同类数量的统计直方图

由于 class 指标是一个分类变量，在后续建立机器学习模型时，必须关注不同类别样本的数量是否平衡。类别不平衡可能会对模型的性能造成不利影响。在机器学习模型的训练过程中，模型可能会倾向于更多样本量的类别，从而忽视样本量较少的类别。这种情况可能导致模型在预测时偏向于预测样本量较多的类别，而对样本量较少的类别的预测效果不佳，从而降低整体分类精度。

(2)连续变量的探索性分析

考虑对单个连续变量进行正态性检验。正态性检验的作用在于验证数据是否符合正态分布的假设。给出部分分子性质指标正态性检验的图像，见图 4：

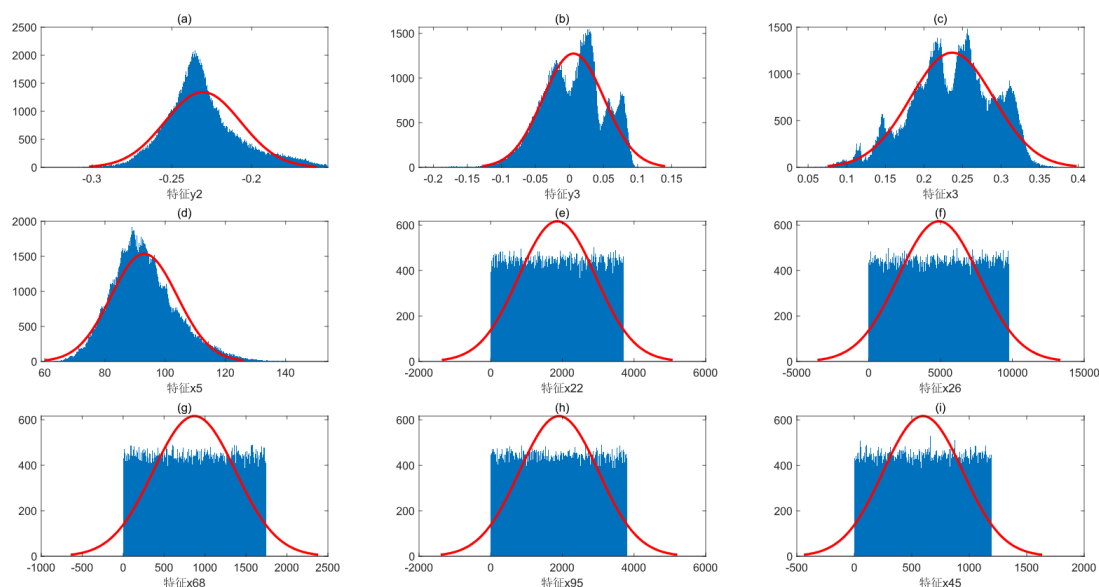


图 4 分子性质指标正态性检验图像（部分）

根据图 4，可知分子性质指标 y_2, y_3, x_5 可近似的看成符合正态分布；分子性质指标 $x_{22}, x_{26}, x_{68}, x_{95}, x_{45}$ 呈现明显的均匀分布； x_3 呈现双峰分布。正态性检验的结果为我们后续在选择相关性分析方法时提供必要的依据。

考虑对绘制两个连续变量，进行散点图的绘制。散点图可以直观展示两个连续变量之间的关系，帮助观察数据之间的分布模式。观察散点图，有助于深入理解数据之间的关联性和规律性。部分分子性质指标间的散点图，见图 5：

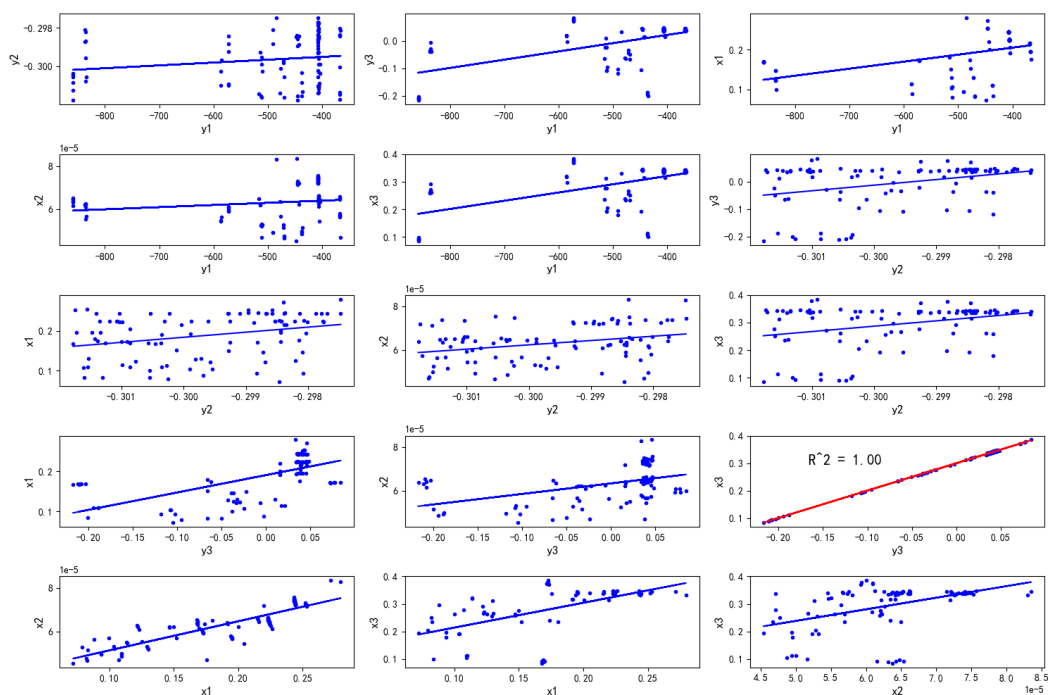


图 5 分子性质指标散点图（部分）

根据图 5，可以看出部分分子性质指标之间的关系，根据图 5 中的红色直线可以看出：分子性质指标 x_3 和 y_3 之间存在显著的线性关系，计算得到其 R^2 达到 1。其余部分分子性质指标的之间的关系在后续数据分析及数学建模过程中得到体现。

5.1.3 直接通过 id 预测 y_2 的模型建立

根据题目要求，需要构建直接通过 id 预测分子性质指标 y_2 的模型，尝试寻找 id 与 y_2 之间的函数关系。

- 绘制 id 与 y_2 之间的散点图

观察图像特征，散点图见图 6：

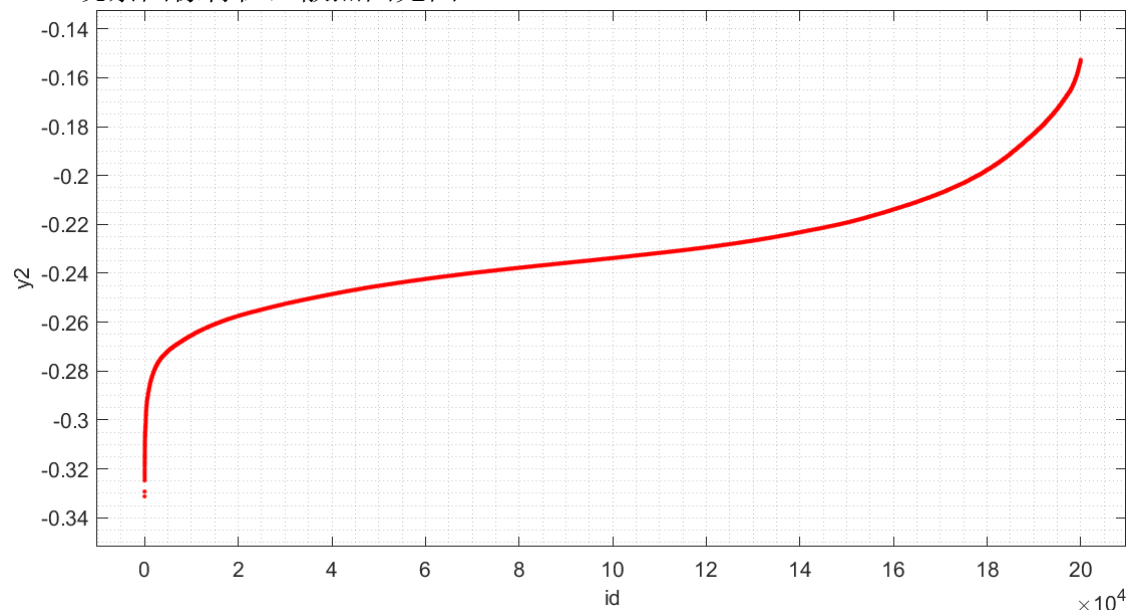


图 6 id 与 y_2 的散点图

- 确定非线性回归模型的拟合函数

根据上述散点图的特征可以直观的观察到 id 与分子性质指标 y_2 的函数关系。考虑分别用如下 3 种拟合方式对函数图像进行拟合：

n 次多项式拟合的表达式，见式(1)：

$$y_2 = a_0 + a_1 x_{id} + a_2 x_{id}^2 + \dots + a_n x_{id}^n \quad (1)$$

n 次高斯函数拟合的表达式，见式(2)：

$$y_2 = a_1 e^{\left(\frac{x-b_1}{c_1}\right)^2} + a_2 e^{\left(\frac{x-b_2}{c_2}\right)^2} + \dots + a_n e^{\left(\frac{x-b_n}{c_n}\right)^2} \quad (2)$$

n 次傅里叶函数拟合的表达式，见式(3)：

$$y_2 = a_0 + a_1 \cos(\omega x_{id}) + b_1 \sin(\omega x_{id}) + \dots + a_n \cos(n\omega x_{id}) + b_n \sin(n\omega x_{id}) \quad (3)$$

考虑用式(1)-(3)的函数对 id 与 y_2 的数据进行拟合，估计参数 $a_i (i=0,1,\dots,n)$ $b_j, c_j (j=1,2,\dots,n)$ ，得到 id 与分子性质指标 y_2 的函数关系。比较上述函数的拟合优度，选取拟合最优的模型。

- 拟合优度的评价指标

考虑使用不同评价指标评价拟合的优度，在这里我们考虑用指标：拟合优度 R^2 、和方差 SSE 、均方差 MSE 及均方根误差 $RMSE$ 四个评价指标对该非线性回归模型的拟合效果进行评价。给出上述指标的计算公式：

R^2 衡量了模型对观测数据的拟合程度，其值越接近 1 表示模型的拟合效果越好。计算公式，见式(4)：

$$R^2 = 1 - \frac{\sum_{i=1}^n (|y_i^* - y_i|)^2}{\sum_{i=1}^n (|\bar{y}_i - y_i|)^2} \quad (4)$$

SSE 反映了拟合数据和原始数据对应点的误差的平方和。计算公式，见式(5)：

$$SSE = \sum_{i=1}^n (y_i^* - y_i)^2 \quad (5)$$

MSE 表示了模型预测的平均偏差程度。计算公式，见式(6)：

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i^* - y_i)^2 \quad (6)$$

$RMSE$ 度量了预测值与实际值之间的平均距离，计算公式，见式(7)：

$$RMSE = \sqrt{\frac{1}{n} (y_i^* - y_i)^2} \quad (7)$$

其中， y_i 表示实测值， $\bar{y}_i$ 表示实测值的均值， y_i^* 表示预测值。

5.1.4 直接通过 id 预测 y_2 的模型求解

对于式(1)-(3)中模型的拟合次数 n 的确定，使用信息准则的方法来选择模型，权衡所估计模型的复杂度和此模型拟合数据的优良性。

本文通过 MATLAB R2023b 平台，研究求解该上述线性回归模型的估计参数 a_i, b_j, c_j ，分别得到三种回归模型的图像和原始数据散点图的图像，见图 7-9：

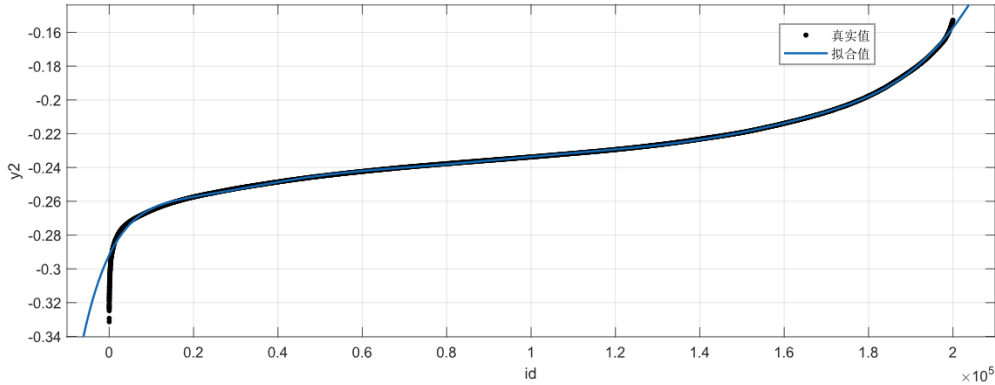


图 7 九阶多项式函数拟合图

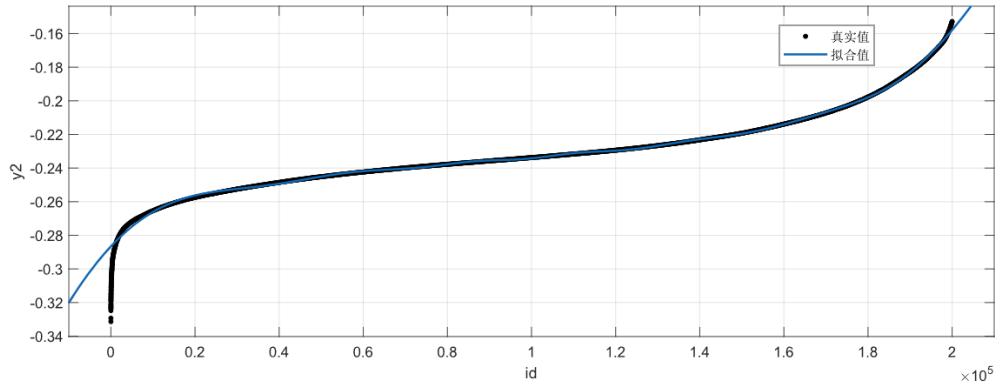


图 8 七阶高斯函数拟合图

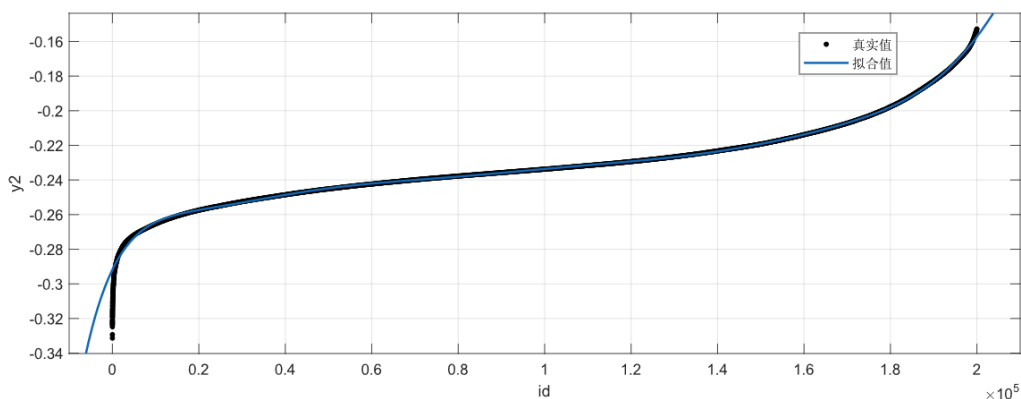


图 9 五阶傅里叶函数拟合图

综合对比分析图 7-9，可直观的看出五阶傅里叶函数的拟合效果较好，拟合的函数与散点图较为贴合。给出上述 3 种非线性回归模型及其拟合优度的评价指标，见表 2：

表 2 非线性回归模型及其评价指标表

非线性回归模型	评价指标			
	R^2	SSE	MSE	$RMSE$
9 阶多项式拟合	0.9987	0.1514	0.000001	0.00086
7 阶高斯函数拟合	0.9977	0.2544	0.000001	0.00111
5 阶傅里叶函数拟合	0.9988	0.1372	0.000001	0.00082

根据表 2，可以直观看到 5 阶傅里叶函数各项评价指标较好，其 R^2 达到 0.9988， SSE 为 0.1372， MSE 为 0.000001， $RMSE$ 为 0.00082 拟合效果最优，与图 6 所得到的结果一致。故考虑用 5 阶傅里叶函数拟合的方式直接呈现 id 与分子性质指标 y_2 的函数关系，见式(8)：

$$\begin{aligned}
 y_2 = & a_0 + a_1 \cos(\omega x_{id}) + b_1 \sin(\omega x_{id}) \\
 & + a_2 \cos(2\omega x_{id}) + b_2 \sin(2\omega x_{id}) + a_3 \cos(3\omega x_{id}) + b_3 \sin(3\omega x_{id}) \\
 & + a_4 \cos(4\omega x_{id}) + b_4 \sin(4\omega x_{id}) + a_5 \cos(5\omega x_{id}) + b_5 \sin(5\omega x_{id})
 \end{aligned}$$

$$\left\{ \begin{array}{l} \omega = 0.000003451 \\ a_0 = -2387000, \\ a_1 = 3711000, b_1 = 1469000 \\ a_2 = -1680000, b_2 = -1576000 \\ a_3 = 374400, b_3 = 793900 \\ a_4 = -12810, b_4 = -199100 \\ a_5 = -6305, b_5 = 195500 \end{array} \right. \quad (8)$$

根据式(8)，通过 predict.csv 中 $id=200000-202580$ 预测分子性质指标 y_2 ，将预测结果填入在 submit.csv 中，我们给出部分非线性回归模型预测结果表，见表 3：

表 3 y_2 的预测结果表（部分）

id	y_2
200001	-0.15388
200002	-0.15386
200003	-0.15387
200004	-0.15386
200005	-0.15385
200006	-0.15385
200007	-0.15384
200008	-0.15383
200009	-0.15383
...	...
202579	-0.13198
202580	-0.13197

5.2 问题二模型建立与求解

5.2.1 特征指标的选取

(1) Spearman 相关性分析

基于数据探索性分析进行的正态分布检验,表明并非所有数据都服从正态分布。Spearman 相关系数法不受数据分布情况的影响。即使数据不符合正态分布假设,仍能有效评估变量之间的关系。基于这一特性,考虑使用 Spearman 相关系数法来评估 $y_2, x_1 \sim x_{100}$ 与分子性质指标 y_1 之间的相关性。

Spearman 相关系数是一种非参数统计量,专门用于衡量两个连续变量之间的单调关系。计算方法是将原始数据转换为等级数据,然后利用排名的计算出等级数据的 Pearson 相关系数来度量它们的相关性。

假设两个随机变量分别为 X 和 Y ,它们的元素个数均为 n , X_i 和 Y_i 分别表示两个随机变量取的第 $i(1 \leq i \leq n)$ 个值。分别对 X 和 Y 中的元素进行排序,得到两个元素排序后的集合 x 和 y ,将排序后集合 x 和 y 中的元素对应相减得到一个排序差分集合 d 。

Spearman 相关系数的计算方式,见式(9):

$$\rho_s = 1 - \frac{6 \sum_{i=1}^n d_i^2}{n(n^2 - 1)} \quad (9)$$

其中, $\rho_s \in [-1, 1]$, $d_i = x_i - y_i (1 \leq i \leq n)$, 元素 x_i 和 y_i 分别为 X_i 在 X 中的排序以及 Y_i 在 Y 中的排序。由于本研究的数据 $n = 20000 \gg 50$, 属于大样本, 检验统计量 t_s 近似服从自由度为 $n - 2$ 的 t 分布。

一般认为 $0.7 < |\rho_s| \leq 1$ 为强相关、 $0.4 < |\rho_s| \leq 0.7$ 为中等强度相关、 $0.2 < |\rho_s| \leq 0.4$ 弱相关、 $0 < |\rho_s| \leq 0.2$ 为极弱相关或无相关。

得到相关系数热力图（部分），见图 10:

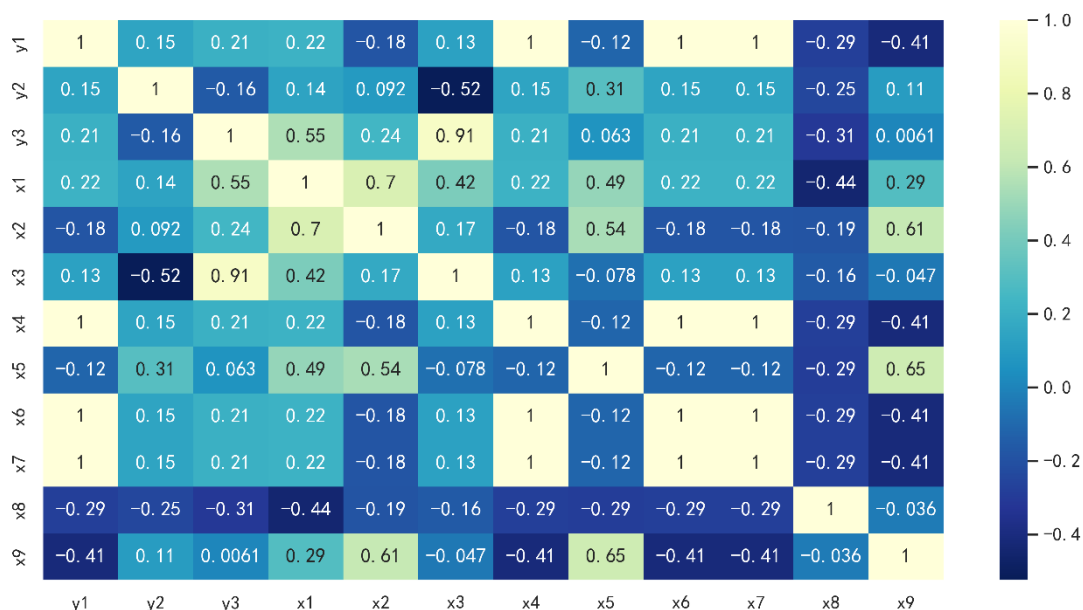


图 10 Spearman 相关性系数热力图（部分）

(2)随机森林特征筛选

随机森林特征筛选法是一种基于随机森林算法的特征选择方法。其原理是通过构建多个决策树组成的随机森林模型，利用特征的重要性评估来选择对目标变量最具影响力的特征，然后根据评估结果筛选出对目标变量有较大贡献的特征。

化学分子数量为 n 个，对 n 中的任意一个样本，有输出值分子性质指标 y_1 及输入值 y_2 和 $x_1 \sim x_{100}$

随机森林模型的特征重要性评估通常采用基于基尼不纯度（Gini impurity, GI）的方法。在随机森林中，每个特征的重要性可以通过计算在每一个树中节点分裂时每个特征的基尼不纯度的平均值来确定。利用特征的重要性选择对目标变量最具影响力的特征。计算 x_i 在第 j 棵决策树上的平均基尼不纯度 GI，见式(10)：

$$GI_j(x_i) = \sum_{i=1}^n P^2 \left(\frac{y}{x_i} \right) - 1 \quad (10)$$

决策树共 N 棵，计算 x_i 在每棵决策树上的 GI，即 $GI_j(x_i) (j=1, 2, \dots, N)$ ，取均值，得到平均值基尼不纯度下降（mean decrease in Gini, MDG），衡量 x_i 的特征重要性，MDG 的计算公式，见式(11)：

$$MDG(x_i) = \frac{1}{N} \sum_{j=1}^N GI_j(x_i) \quad (11)$$

选取 MDG 的值较高的指标，见表 4：

表 4 特征变量及其 MDG

指标名称	MDG	排名
x_4	0.890	1
x_6	0.881	2
x_7	0.858	3
x_{23}	0.656	4
x_{33}	0.515	5

(3)综合上述模型的指标筛选

Spearman 相关系数提供了单变量相关性的直接评估，而随机森林则通过综合多棵决策树的结果提供特征重要性的整体评估。结合两种方法，我们可以更好地筛选出真正有意义的特征，减少误选或漏选重要特征的风险。这样不仅提高了模型的准确性和稳健性，也为后续的数据分析和建模奠定了坚实的基础。

综上，我们选取分子性质指标 x_4, x_6, x_7 作为模型的输入指标，预测分子性质指标 y_1 。

5.2.2 机器学习回归模型的建立

通过预测模型的建立与求解，我们选择使用机器学习模型进行建模。这种方法在处理大数据方面具有显著优势，能够高效地处理海量数据集。此外，机器学习模型在多指标数据处理上表现出强大的能力，能够自动识别和选择重要特征，并有效地处理高维数据。相比传统统计方法，机器学习模型在处理复杂数据特性，如非线性关系和噪声数据方面，也表现得更加出色。

机器学习模型在大数据、多指标和复杂数据处理方面展现了卓越的性能。通过使用这些模型，我们能够建立更准确、更鲁棒的预测模型，提高模型的预测性能和泛化能力。

(1)决策树（回归树）模型建立

决策树是一种基于树状结构的监督学习算法，其核心思想是分层决策。决策树将数据分割成不同的子集，以树状结构表示决策规则。每个节点代表一个特征测试，每个叶子节点代表一个目标变量的预测结果。在构建回归树时，算法通过选择最佳特征来分裂数据，并递归地构建子树。决策树模型易于解释，因为它们可以用树形图表示决策路径。然而，它们在某些情况下容易过拟合，因此可以采用剪枝等技术改善性能。

给出决策树模型的原理如下：

假定样本数据集为 $(x_i, y_i)(i=1,2,3)$ ，其中 x_i 为特征指标筛选得到的分子样本指标输入； y_i 为需要预测的指标，作为输出。

回归树通过不断二分递归构建二叉树，选取第 j 个解释变量 x^j 及其对应值 s 作为切分变量和切分点，将训练集划分为两个子区域：

$$R_1(j, s) = \{x | x^j \leq s\}, R_2(j, s) = \{x | x^j > s\} \quad (12)$$

得到相应的输出值为：

$$c_m^* = \frac{1}{n_m} \sum_{x_m \in R_m(j, s)} y_i, m=1,2 \quad (13)$$

遍历解释变量 j 和及其所有可能的值 s ，当 j 和 s 满足下式是为最优的切分变量和最优切分点

节点的损失函数的形式：

$$\min_{j,s} \left[\min_{c_1} \text{Loss}(y_i, c_1) + \min_{c_2} \text{Loss}(y_i, c_2) \right] \quad (14)$$

节点有两条分支， c_1 是左节点的平均值， c_2 是右节点的平均值，将划分的两个区域递归的调用式(12)进行不断的划分，每一次划分都是使得划分出的两个分支的误差和最小，直到满足停止条件为止。最

最终训练集被划分为 L 个叶节点 $R_1, R_2, \dots, R_L$ ，每个叶节点上的相应变量值为：

$$\bar{y}_l = \{\bar{y} | x_i \in R_l, l=1,2,\dots,L\} \quad (15)$$

给出决策树模型的伪代码：

算法 1：决策树

输入：训练集 $D=\{(x_1,y_1),(x_2,y_2),\dots,(x_m,y_m)\}$;

属性集 $A=\{a_1,a_2,\dots,a_d\}$

过程：函数 TreeGenerate(D,A)

1: 生成结点 node;

2: if D 中样本全属于同一类别 C then

3: 将 node 标记为 C 类叶节点; return

4: end if

5: if $A=\varnothing$ OR D 中样本在 A 上取值相同 then

6: 将 node 标记为叶节点,其类别标记为 D 中样本数最多的类; return

7: end if

8: 从 A 中选择最优划分属性 a_* ;

9: for a_* 的每一个值 a_*^v do

10: 将 node 生成一个分支; 令 D_v 表示在 D 中在 a_* 上取值为 a_*^v 的样本子集;

11: if D_v 为空 then

12: 将分支结点标记为叶节点, 其类别标记为 D 中样本最多的类; return

13: else

14: 以 TreeGenerate($D_v, A \setminus a_*$) 为分支节点

15: end if

16: end for

输出：以 node 为根结点的一棵决策树

(2)XGBoost 模型建立

XGBoost 是一种基于梯度提升树的高性能算法,具有较高的准确性和可扩展性。该算法能够处理大规模的数据集,并且可以在多个 CPU 上并行运行,具有高效性和可扩展性的优势。XGBoost 的核心思想是通过一系列的弱分类器(通常是决策树)来构建一个强大的分类器。在每个迭代中, XGBoost 会寻找最佳的分类器来提高模型的准确性,这个过程是通过计算每个分类器的梯度来实现的。在 XGBoost 模型中,每个决策树都由一个由节点组成的集合构成,一个特征和一个阈值组合成一个节点,用来将数据分成两份。在训练过程中, XGBoost 会计算每个节点的梯度,并使用这些梯度来选择最佳的特征和阈值。然后, XGBoost 将数据分成两个子集,并继续递归地构建树。在构建树的过程中, XGBoost 还使用了一些正则化技术,例如 L1 正则化和 L2 正则化,目的是让模型的过拟合风险降低,同时增强模型的泛化能力。除了使用决策树来构建模型之外, XGBoost 还使用了自适应学习率技术,以提高模型的训练速度和准确性。

假定样本数据集为 $(x_i, y_i)(i=1,2,3)$, 其中 x_i 为特征指标筛选得到的分子样本指标输入; y_i 为需要预测的指标,作为输出。XGBoost 可以同时考虑模型的预测能力和运行效率,是一种非常实用且有效的机器学习算法。其目标函数如下:

$$Obj^{(t)} = \sum_{i=1}^n l(y_i, y_i^{(t-1)} + f_i(x_i)) + \Omega(f_i) + C \quad (16)$$

对于本文的分子指标特征数据集, i 就代表第 i 个汽车样本。模型的目标函数由两部分组成,第一部分是衡量预测值与真实值之间差异的损失函数,选择损失

函数应根据需求确定。第二部分是正则化项，用树模型的某种变换 Ω 表示，用于衡量模型的复杂度。

二阶泰勒展开式为：

$$f(x + \Delta x) \approx f(x) + f'(x)\Delta x + \frac{1}{2} f''(x)\Delta x^2 \quad (17)$$

对损失函数二阶泰勒展开：

$$Obj^{(t)} = \sum_{i=1}^n l(y_i, y_i^{(t-1)}) + g_i f_i(x_i) + \frac{1}{2} h_i f_i^2(x_i) + \Omega(f_t) + \text{const} \quad (18)$$

XGBoost 是一种使用二阶偏导泰勒展开方法来优化模型性能的机器学习算法。该方法能够提高梯度下降的速度和准确性，而且不需要确定具体的损失函数形式，只需要输入数据即可计算损失函数。这种方法使用自变量的二阶导数形式，在泰勒展开函数中进行计算，从而实现更高效的模型优化。这种形式的损失函数使得模型更加通用，适用于各种数据集。通过使用二阶偏导泰勒展开方法，XGBoost 能够更有效地处理复杂和大规模的数据集，从而获得更好的预测结果。

给出 XGboost 模型的伪代码：

算法 2: XGboost

输入：当前节点的实例集 I 特征维数 d

输出：最大得分

```

1:  $gain \leftarrow 0$ 
2:  $G \leftarrow \sum_{i \in I} g_i, H \leftarrow \sum_{i \in I} h_i$ 
3: for  $k=1$  to  $m$  do
4:    $G_L \leftarrow 0, H_L \leftarrow 0$ 
5:   for  $j$  in sorted( $I$ , by  $X_{jk}$ ) do
6:      $G_L \leftarrow G_L + g_j, H_L \leftarrow H_L + h_j$ 
7:      $G_R \leftarrow G - G_L, H_R \leftarrow H - H_L$ 
8:      $score \leftarrow \max(score, \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda})$ 
9:   end
10: end

```

(3)支持向量机回归模型建立

支持向量机回归（Support Vector Regression, SVR）是一种用于回归分析的机器学习方法。它基于支持向量机（SVM）的原理，通过引入 ϵ -不敏感损失函数，允许预测值与实际值之间有一个 ϵ 范围内的误差，目标是在这个误差范围内找到一个最优的回归超平面。SVR 同时最小化模型复杂度和预测误差，适用于高维数据。

给出 SVR 模型的原理如下：

假定样本数据集为 $(x_i, y_i)(i=1, 2, 3)$ ，其中 x_i 为特征指标筛选得到的分子样本指标输入； y_i 为需要预测的指标，作为输出。

构建在高维特征空间中的线性回归函数，见式(12)：

$$f(x) = \omega^T \varphi(X) + b \quad (19)$$

其中 $\varphi(x)$ 表示非线性映射函数， ω 为权向量， b 为阈值。 ω 和 b 都是参数。

引入一个 ϵ -不敏感损失函数

$$L(y, f(x)) = L(|y - f(x)|) = \begin{cases} 0, & |f(x) - y| \leq \varepsilon \\ |f(x) - y| - \varepsilon, & |f(x) - y| > \varepsilon \end{cases} \quad (20)$$

其中 $f(x)$ 表示回归函数得到的预测值, ε 代表误差区间, y 为相应的预测值。

SVR 问题可表示为:

$$\min_{\omega, b} \frac{1}{2} \|\omega\|^2 + C \sum_{i=1}^m L(|y_i - f(x_i)|) \quad (21)$$

引入松弛变量转化上述凸优化问题:

$$\begin{aligned} \min_{\omega, b, \xi_i, \xi_i^*} \quad & \frac{1}{2} \|\omega\|^2 + C \sum_{i=1}^m (\xi_i + \xi_i^*) \\ \text{s.t.} \quad & \begin{cases} y_i - \omega^T(x_i) - b \leq \varepsilon + \xi_i \\ -y_i + \omega^T(x_i) + b \leq \varepsilon + \xi_i^* \\ \xi_i \geq 0, \xi_i^* \geq 0, \end{cases} \end{aligned} \quad (22)$$

其中 C 代表惩罚系数, 随着 C 的增加, 意味着对训练误差大于 ε 的样本的惩罚增加, ε 表示回归系数的误差, ε 越小意味着回归函数的误差就会越小。

引入拉格朗日乘子 α 和 η , 转换后的目标函数:

$$\begin{aligned} L(\omega, b, \xi_i, \xi_i^*) = & \frac{1}{2} \|\omega\|^2 + \gamma \sum_{i=1}^m (\xi_i + \xi_i^*) - \sum_{i=1}^m \alpha_i (\varepsilon + \xi_i + y_i - \omega x_i - b) \\ & - \sum_{i=1}^m \alpha_i^* (\varepsilon + \xi_i^* + y_i - \omega x_i - b) - \sum_{i=1}^m (\eta_i \xi_i + \eta_i^* \xi_i^*) \end{aligned} \quad (23)$$

函数求偏导数并让其为 0, 非线性映射 φ 可以通过将样本数据映射到一个更高维空间, 来解决非线性问题, 再采用适当的核函数代替高维空间中的内积向量, 得到最终的 SVR 回归函数方程:

$$f(x) = \sum_{i=1}^n \sum_{j=1}^n (\alpha_i - \alpha_i^*) \kappa(x_i, x) + b \quad (24)$$

其中, $\kappa(x_i, x)$ 表示的是支持向量机回归的核函数。

SVR 是以统计学理论为基础, 针对凸二次优化问题进行求解的一种机器学习方式。可以在解决传统机器学习方法陷入局部极值问题的基础上获得全局最优解。VC 维结构和风险最小化准则是 SVR 的基础, SVR 算法在控制机器学习机的复杂程度和协调风险最小化的同时, 实现解决算法“过学习”的目的。而且在针对小样本数据或有限样本数据, SVR 模型学习能力较强并且具有良好的泛化能力。

(4)BP 神经网络模型建立

反向传播神经网络 (BPNN) 是一种用于回归任务的多层前馈神经网络。BP 神经网络学习算法是误差反向传播, 属于多层前馈网络, 采用梯度下降法, 达到预测最佳值, 具有处理非线性信息的能力。BP 神经网络包含输入层、隐藏层和输出层三层结构, 它的实现原理是通过信息正向传播和误差反向传播的循环, 从而不断调整每一层的权重和阈值, 最终在网络的总误差 E 小于预先设定的最大误差或达到预定的训练次数时终止。一般的神经网络图, 见图 11:

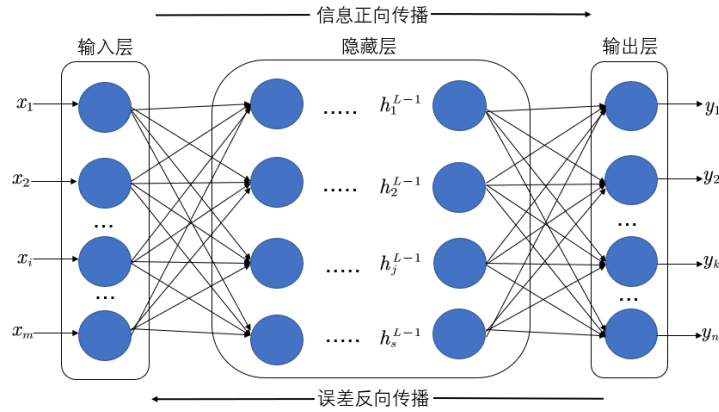


图 11 Bp 神经网络结构图

给出 Bp 神经网络的原理如下：

假定样本数据集为 $(x_i, y_i)(i=1,2,3)$ ，其中 x_i 为特征指标筛选得到的分子样本指标输入； y_i 为需要预测的指标，作为输出。

构建一个神经网络。分为如下几个步骤：信号正向传播；误差反向传播；更新权值与阈值；保存训练网络。

(a)信号正向传播

将学习样本 $p^k = (x_1^k, \dots, x_n^k)$ 即 id=0-20000 的指标 x_i 输入模型，其中 x_n^k 表示样本 k 中的输入神经元 n。

使用输入样本数据、连接权重和阈值依次计算各层神经元的输出值：

$$Y_j = f\left(\sum_{i=1}^n \omega_{ij} X_i - \theta_j\right) \quad (25)$$

$$Z_l = f\left(\sum_{i=1}^n V_{li} Y_i - \theta_l\right) \quad (26)$$

其中， Y_j 表示网络隐含层节点； Z_l 表示网络输出层节点； ω_{ij} 表示输入层节点与隐含层节点之间的权值； V_{li} 表示隐含层节点与输出层节点之间的权值； θ_j 表示隐含层阈值； θ_l 表示输出层阈值； f 函数为神经元的激活函数，实际操作中一般选用 Sigmoid 型函数，其计算公式为：

$$f(x) = \frac{1}{1 + e^{-x}} \quad (27)$$

(b)误差反向传播

输出层神经元的学习误差 δ_l 和隐含层神经元的学习误差 δ_j 的计算公式：

$$\delta_l = -(T_l - Z_l) f'\left(\sum_{i=1}^n V_{li} Y_i - \theta_l\right) \quad (28)$$

$$\delta_j = f'\left(\sum_{i=1}^n \omega_{ij} X_i - \theta_j\right) \sum_{l=1}^n \delta_l V_{lj} \quad (29)$$

其中， T_l 为输出层节点的期望输出； f' 为神经元激活函数的导函数。

(c)更新权值与阈值

使用标准均方误差函数来计算模型样本的相对误差：

$$E = \sum \frac{1}{2} (T_l - Z_l) \quad (30)$$

若计算出的模型误差 E 小于给定的学习精度 ε ，则说明形成的函数关系符合测算要求；反之，则需要对模型各层之间的权值和阈值进行调整。

隐含层到输出层之间的权值和阈值调整方式为：

$$V_{ij}(k+1) = V_{ij}(k) + \Delta V_{ij} = \eta \delta_l Y_j + V_{ij}(k) \quad (31)$$

$$\theta_l(k+1) = \theta_l(k) + \eta \delta_l \quad (32)$$

输入层到隐含层之间的权值和阈值调整方式为：

$$W_{ij}(k+1) = W_{ij}(k) + \Delta W_{ij} = \eta \delta_j X_j + W_{ij}(k) \quad (33)$$

$$\theta_j(k+1) = \theta_j(k) + \eta \delta_j \quad (34)$$

调整权值和阈值后，重新进行信息正向传播过程和误差反向传播过程，直到中模型误差 E 小于学习精度 ε 为止。

(d)保存训练网络。

保存上述训练完成的神经网络模型。运用保存下来的稳定函数关系模型对已有数据进行训练和学习。

从输入数据到输出结果，BP神经网络模型实质上是完成两者之间的一个非线性映射。一个三层的BP神经网络就能够以任意精度逼近任何非线性连续函数，这意味着网络具有较强的非线性映射能力，该模型也因此被广泛用于处理内部机制复杂的实际问题。对于不在原有训练样本中的数据，BP神经网络也能够利用训练好的网络对其进行拟合，泛化能力较强。

给出神经网络的算法如下：

算法 3：神经网络

输入： 训练集 $D = \left\{ (x^{(n)}, y^{(n)}) \right\}_{n=1}^N$ ，验证集 V ，学习率 α ，正则化系数 λ ，网络层数 L ，神经元数量 M_l ， $1 \leq l \leq L$ 。

输出： W, b

- 1: 随机初始化 W, b ;
 - 2: repeat
 - 3: 对训练集 D 中的样本随即重排序;
 - 4: for $n = 1 \cdots N$ do
 - 5: 从训练集 D 中选取样本 $(x^{(n)}, y^{(n)})$;
 - 6: 前馈计算每一层的净输入 $z^{(l)}$ 和激活值 $a^{(l)}$ ，直到最后一层;
 - 7: 反向传播计算每一层的误差 $\delta^{(l)}$;
 - 8: $\forall l, \frac{\partial \zeta(y^{(n)}, y^{(n)})}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T$;
 - 9: $\forall l, \frac{\partial \zeta(y^{(n)}, y^{(n)})}{\partial b^{(l)}} = \delta^{(l)}$;
 - 10: $W^{(l)} \leftarrow W^{(l)} - \alpha \left(\delta^{(l)} (a^{(l-1)})^T + \lambda W^{(l)} \right)$;
 - 11: $b^{(l)} \leftarrow b^{(l)} - \alpha \delta^{(l)}$;
 - 12: end
 - 13: until 神经网络模型在验证集 V 上的错误率不再下降;
-

(5)机器学习回归模型评价指标

考虑使用如下指标评价机器学习回归模型的优劣：在这里我们考虑用指标：拟合优度 R^2 、平均绝对误差 MAE 及均方根误差 $RMSE$ 三个评价指标对该机器学习回归模型的拟合效果进行评价。给出上述指标的计算公式：

R^2 衡量了模型对观测数据的拟合程度，其值越接近 1 表示模型的拟合效果越好。计算公式，见式(35)：

$$R^2 = 1 - \frac{\sum_{i=1}^n (|y_i^* - y_i|)^2}{\sum_{i=1}^n (|\bar{y}_i - y_i|)^2} \quad (35)$$

MAE 反映了预测值与实际值之间的平均距离。计算公式，见式(36)：

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i^* - y_i| \quad (36)$$

$RMSE$ 度量了预测值与实际值之间的平均距离，计算公式，见式(37)：

$$RMSE = \sqrt{\frac{1}{n} (y_i^* - y_i)^2} \quad (37)$$

其中， y_i 表示实测值， $\bar{y}_i$ 表示实测值的均值， y_i^* 表示预测值。

5.2.3 机器学习回归模型的求解

在上述机器学习模型中，上述模型都需要对输入数据进行归一化处理。

机器学习中，模型参数的重要性不可忽视。模型参数包括在训练过程中通过数据学习得到的模型参数和需要预先设置的超参数。正确选择和优化这些参数可以显著提升模型的性能和预测能力。通过参数调优，确保模型在实际应用中能够准确、稳定地进行预测，是构建高效机器学习模型的关键步骤。我们给出多种机器学习模型的参数设置，见表 5：

表 5 机器学习模型及其参数设置

模型	超参数设置
DT	MaxDepth: 15
	MinLeafSize: 2
	SplitCriterion: MSE
	SplitMethod: MDL
BPNN	HiddenLayerSize: 5
	LearningRate: 0.01
	TrainFcn: traingd
	TransferFcn: sigmoid
	MaxEpochs: 100
SVM	Gamma: 0.75
	C: 4.0
	Epsilon: 0.1
XGBoost	LearningRate: 0.015
	MaxDepth: 10
	Lambda: 0.1
	TreeNum: 50
	ColsampleBylevel: 0.5

选取指标 x_4, x_6, x_7 作为机器学习模型的输入指标，以 y_1 的值作为机器学习模型的输出。得到多种机器学习模型在训练集和测试集上的表现，见表 6：

表 6 多种机器学习模型在训练集和测试集上的表现

模型	训练集			测试集		
	R^2	MAE	$RMSE$	R^2	MAE	$RMSE$
DT	0.99956	0.55834	0.84422	0.99955	0.56141	0.83957
BPNN	0.99978	0.26825	0.41879	0.99976	0.27015	0.44538
SVR	0.99736	5.59520	6.42456	0.99739	5.61868	6.44951
XGBoost	0.99961	0.51941	0.78209	0.99964	0.51946	0.75519

展示最优模型：Bp 神经网络的求解结果，给出其训练迭代过程，见图 12：

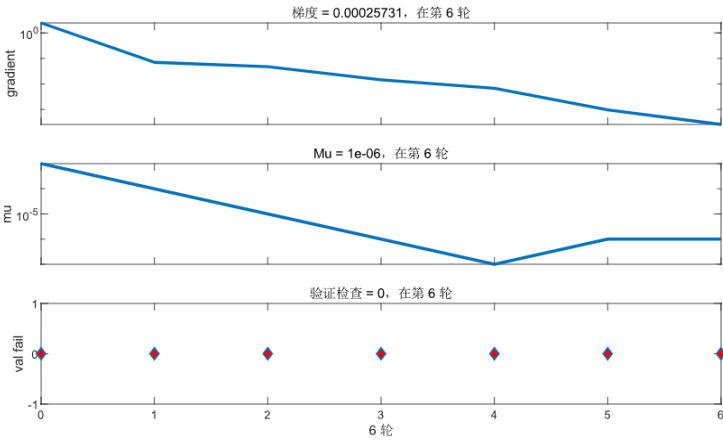


图 12 Bp 神经网络模型的迭代过程

根据图 12，随着迭代的进行，梯度和学习率都在减小，为了使网络收敛到一个稳定的解。验证检查为 0 意味着在第 6 轮迭代时，模型在验证集上的性能已经达到了一个满意的水平，或者模型的复杂度得到了控制。在 BP 神经网络的训练过程中，监控这些参数是非常重要的，因为它们可以帮助我们了解网络的学习进度，调整学习率，以及决定何时停止训练以避免过拟合。

给出 Bp 神经网络的训练结果图，见图 13：

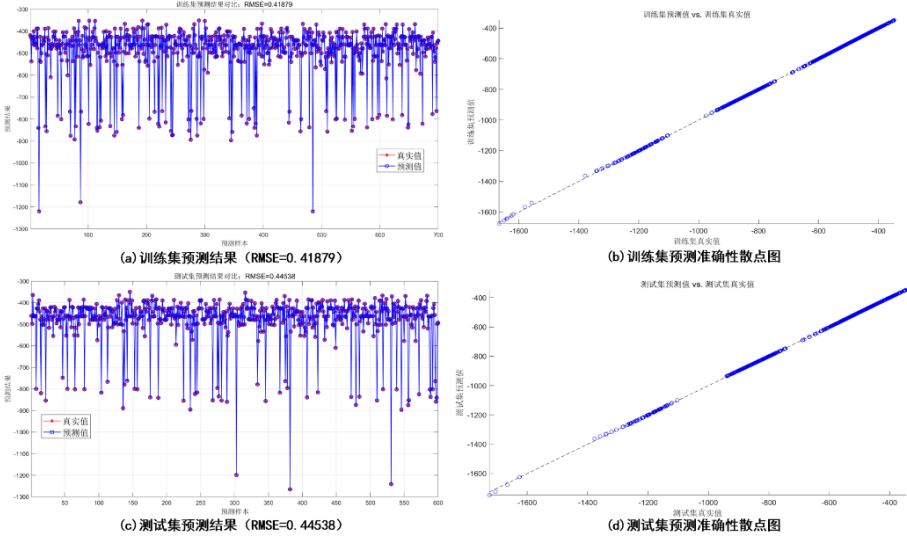


图 13 Bp 神经网络在训练集和测试集上的预测效果

根据图 13，模型在训练集上的预测表现良好，RMSE 为 0.41879。散点图显示实际值和预测值之间有很高的相关性。模型在测试集上的预测表现也很好，RMSE 为 0.44538。尽管比训练集稍差，但散点图同样显示出很高的相关性。从训练集和测试集的低 RMSE 值可以看出，该模型在预测上的准确性很高。虽然测试集的 RMSE 稍高于训练集，但差异不大，说明模型在处理未见数据时表现稳定，模型泛化能力较好

根据选取的最优模型 Bp 神经网络训练结果，通过 predict.csv 中相关指标预测 y_1 ，将预测结果填入在 submit.csv 中，我们给出 y_1 的部分预测结果表，见表 7：

表 7 y_1 的预测结果表（部分）

id	y_1
200001	-368.7206
200002	-384.7827
200003	-367.4898
200004	-384.7665
200005	-384.7847
200006	-387.6592
200007	-367.5808
200008	-368.7328
...	...
202579	-477.0412
202580	-519.1188

5.3 问题三模型的建立与求解

5.3.1 特征指标的选取

和问题二一致，我们考虑采用Spearman相关性分析以及随机森林特征选择的方法进行特征筛选。

直接给出Speraman相关系数图（部分），见图14：

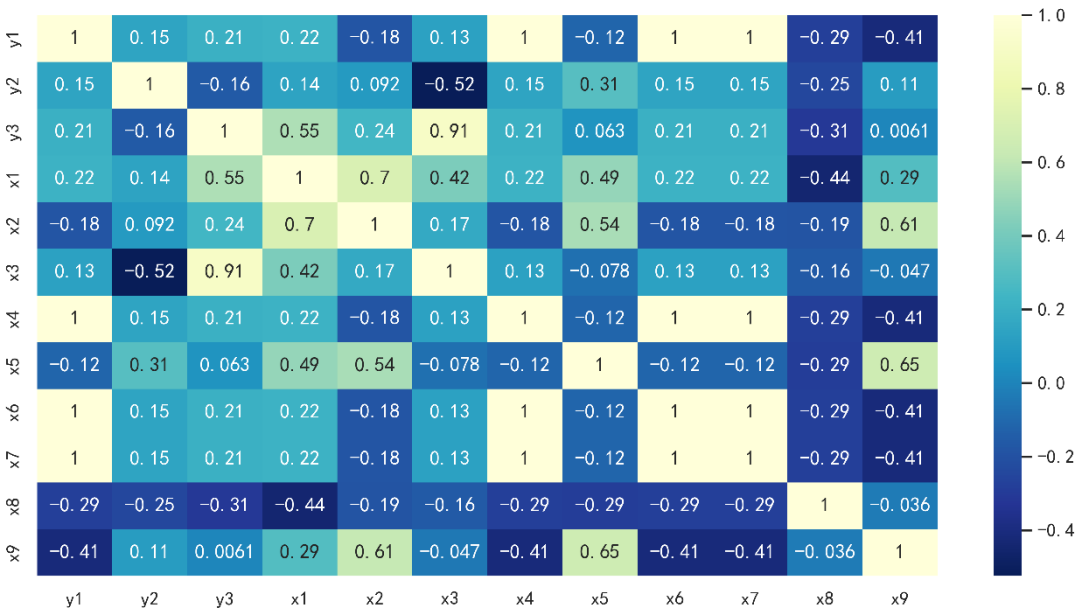


图 14 Speraman 相关系数图（部分）

随机森林特征筛选MDG表（部分），见表8：

表 8 特征变量及其 MDG

指标名称	MDG	排名
x_3	0.970	1
x_1	0.581	2
x_2	0.368	3
y_1	0.156	4
x_6	0.115	6
x_4	0.107	7
x_7	0.103	8
x_{11}	0.031	9

通过综合使用 Spearman 相关系数法和随机森林特征选择法，我们能够更全面地筛选出对目标变量最有影响力的特征。Spearman 相关系数提供了单变量相关性的直接评估，而随机森林则通过综合多棵决策树的结果提供特征重要性的整体评估。结合两种方法，我们可以更好地筛选出真正有意义的特征，减少误选或漏选重要特征的风险。这样不仅提高了模型的准确性和稳健性，也为后续的数据分析和建模奠定了坚实的基础。

综上，我们选取指标 x_1, x_3 作为模型的输入指标，预测 y_3 。

5.3.3 函数拟合模型的建立与求解

基于特征指标选取得到两个指标，为具体分析因变量 y_3 和自变量 x_1, x_3 之间的函数关系，本文考虑对其进行多项式回归，通过评估函数拟合优度，得到预测效果最好的模型，构建四个多项式回归模型，见式(38)-(41)：

(1)线性

$$y_1 = \beta_0 + \beta_1 x_1 + \beta_2 x_3 \quad (38)$$

(2)纯二次型

$$y_2 = \beta_0 + \beta_1 x_1 + \beta_2 x_3 + \beta_3 x_1^2 + \beta_4 x_3^2 \quad (39)$$

(3)交叉型

$$y_3 = \beta_0 + \beta_1 x_1 + \beta_2 x_3 + \beta_3 x_1 x_3 \quad (40)$$

(4)完全二次型

$$y_4 = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1^2 + \beta_4 x_3^2 + \beta_5 x_1 x_3 \quad (41)$$

其中 β_0 为常量， $\beta_i (i=1, 2, \dots, n)$ 表示自变量系数。

对上述四个模型进行求解，可得模型具体表达式，见式(42)-(45)：

(1)线性

$$y = -0.206 + 0.287 x_1 + 0.670 x_3 \quad (42)$$

(2)纯二次型

$$y = -0.257 + 0.920 x_1 + 0.610 x_3 - 1.691 x_1^2 + 0.129 x_3^2 \quad (43)$$

(3)交叉型

$$y = -0.257 + 0.550 x_1 + 0.892 x_3 - 1.144 x_1 x_3 \quad (44)$$

(4)完全二次型

$$y = -0.268 + 0.933x_1 + 0.685x_2 - 1.302x_1^2 + 0.269x_3^2 + -0.710x_1x_3 \quad (45)$$

本文选取 R^2 、 $RMSE$ 、 P 值作为拟合效果的评价指标，对上述四个回归模型进行模型比较，结果见表 9：

表 9 回归模型比较

回归模型	R^2	$RMSE$	P 值
线性	0.8548	0.01710	1.7803E-14
纯二次型	0.8694	0.01682	6.3128E-14
交叉型	0.8778	0.01692	2.8914E-13
完全二次型	0.9014	0.01578	4.2156E-14

其中 R^2 表示拟合优度， $RMSE$ 表示均方根误差， P 值表示接受模型的风险。

由表 9 可知，完全二次型回归模型的拟合效果优于其余回归模型。完全二次型相较于线性、纯二次型、交叉型， R^2 分别提高 0.0466、0.032 、0.0236， $RMSE$ 分别降低 0.00132、0.00104、0.00114。

对上述模型拟合效果进行可视化，见图 15：

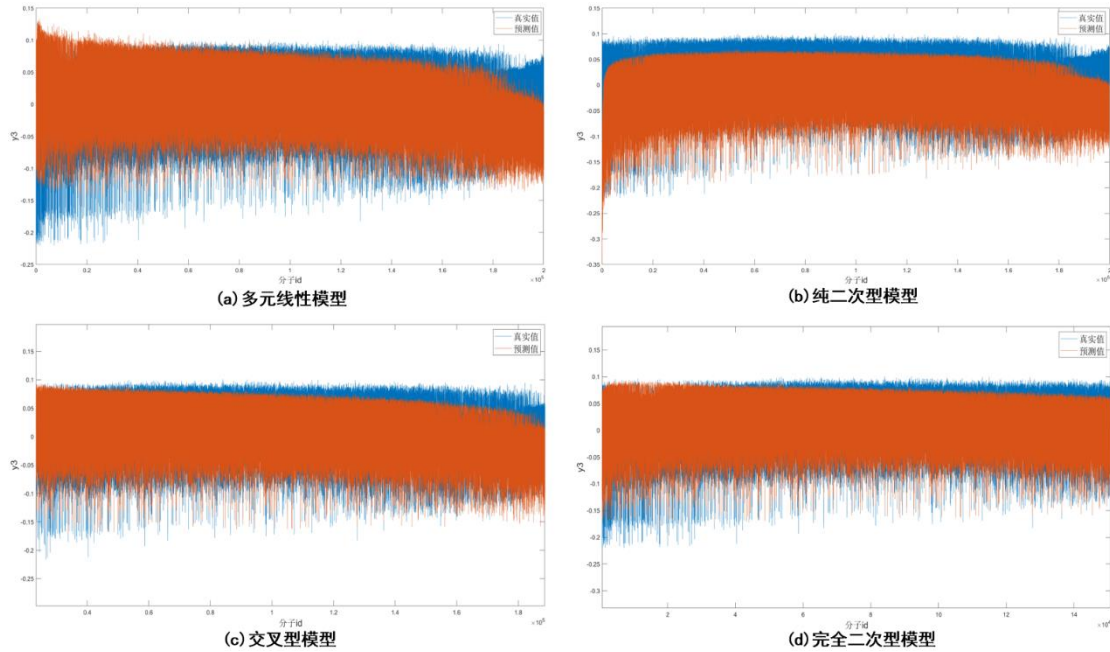


图 15 四种回归模型对 y_3 的拟合效果比较

观察图 15，完全二次型模型对 y_3 的整体拟合效果总体较好，基本没有出现较大的误差波动，且模型变量少，对于 y_3 具有很好的可解释性。

通过逐步回归分析，我们发现在完全二次型回归模型中不存在无关变量。这意味着所有选择的自变量在模型中都对因变量的解释有重要贡献，没有冗余变量干扰预测结果，最终关于 y_3 的函数关系为：

$$y = -0.268 + 0.933x_1 + 0.685x_2 - 1.302x_1^2 + 0.269x_3^2 + -0.710x_1x_3 \quad (46)$$

5.3.4 函数拟合精度误差分析

为了解该函数对于 y_3 的拟合精度好坏，本文考虑选用绝对误差对其进行误差分析。绝对误差是预测值与实际值的差值，它能够反映预测值偏离实际值的大小，具体见图 16：

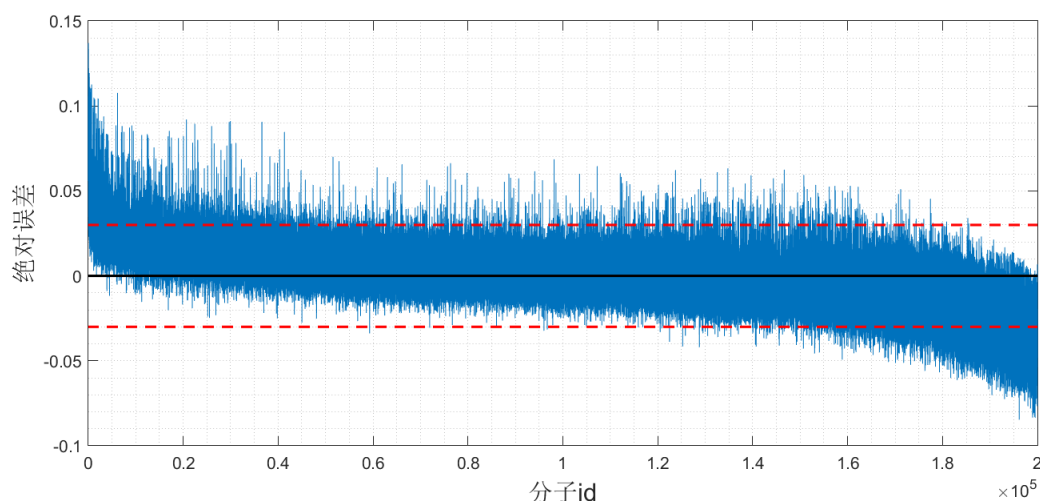


图 16 模型误差分析图

观察图 16，完全二次型函数拟合的绝对误差基本保持在 30%波动之内，且绝对误差基本在前部和后部出现较大的偏差。因此，可以认为该完全二次型回归模型具有不错的拟合效果。结果说明，本文选择的自变量 x_1 ， x_3 能够很好地解释因变量 y_3 的变动，并且在调整后的模型中没有遗漏或不必要的变量。

5.3.5 灵敏度分析

基于上述完全二次型回归模型，我们考虑对 x_1 和 x_3 进行灵敏度分析，通过对自变量进行微小波动来观察因变量 y_3 的变化，具体实现步骤见下：

Step1:选择 x_1 或 x_3 ，将其值进行上下波动 5%

Step2:使用调整后的自变量值重新计算预测值

Step3:通过原预测值减新预测值，计算预测值的变化量，即预测值差异。

Step4:将预测值变化量除以自变量的变化值，求得灵敏度指标

根据上述步骤所得的灵敏度指标，表示每单位自变量的变化对因变量 y_3 的影响程度。

对 x_1 和 x_3 分别进行上下波动 5%，得到如下预测值变化图，见图 17：

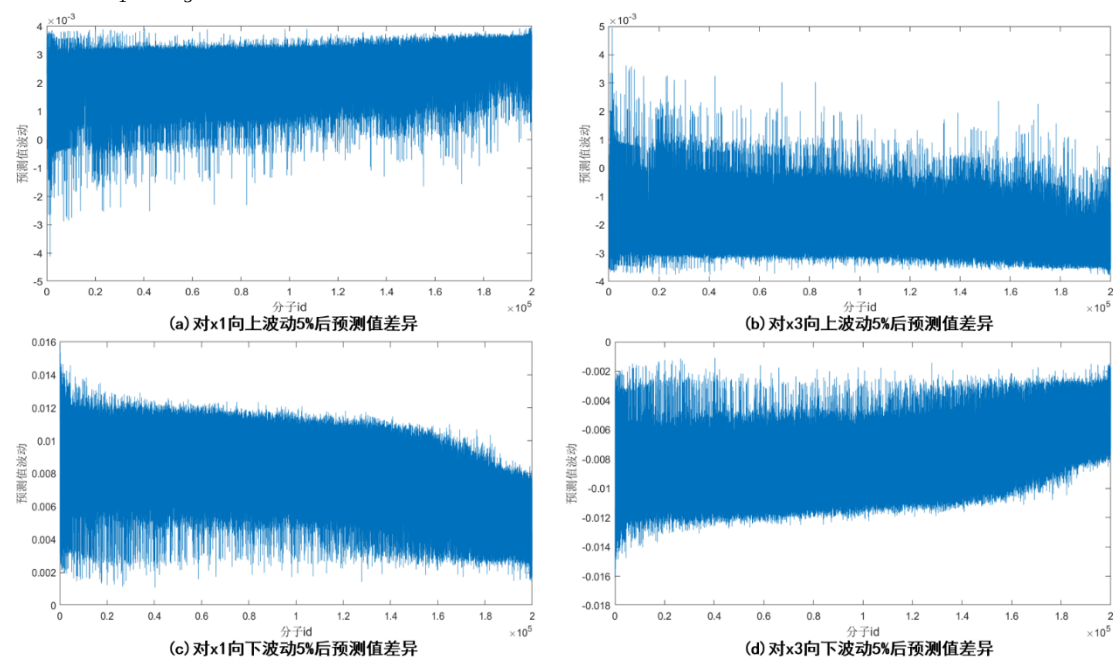


图 17 自变量波动对预测值的影响

由图 17 (a) (c) 可知, 对 x_1 进行上下波动 5% 后, 新预测值会整体偏低, 因此 y_3 与 x_1 呈负敏感性; 由图 17 (b) (d) 可知, 对 x_3 进行上下波动 5% 后, 新预测值会整体偏高, 因此 y_3 与 x_3 呈正敏感性。

对预测值变化量与自变量变化量作商, 得到如下灵敏度指标, 见表 10:

表 10 灵敏度指标表

y_3 对 x_1 的灵敏度	y_3 对 x_3 的灵敏度
-12.63	10.54

通过上述灵敏度指标可得, y_3 对 x_1 具有较高的负敏感性, y_3 对 x_3 具有较高的正敏感性。

5.4 问题四模型的建立

5.4.1 特征指标选取

(1) 互信息法

在信息论中, 信息熵被用来描述信号源的不确定度。设有一个离散变量 X , 它的值域为 Ω , X 的概率质量函数 (Probability Mass Function, PMF) $p(x) = \Pr\{X = x\}, x \in \Omega$ 。这是定义随机变量 X 的信息熵 $H(X)$ 为:

$$H(X) = -\sum_{x \in \Omega} p(x) \log_b p(x) = \sum_{x \in \Omega} p(x) \log_b \frac{1}{p(x)} \quad (47)$$

一个随机变量的信息熵可以用公式表示, 那么一对随机变量也可以通过推导得到他们的联合熵 (Joint Entropy, JE)。设有一对离散随机变量 (X, Y) , 他们的联合分布为 $p(x, y)$, 那它们的联合熵为式(48):

$$H(X, Y) = -\sum_{x \in \Omega} \sum_{y \in \Omega} p(x, y) \log p(x, y) \quad (48)$$

且联合熵符合:

$$\max\{H(X), H(Y)\} \leq H(X, Y) \leq H(X) + H(Y) \quad (49)$$

条件熵的定义如式(50)所示:

$$H(X|Y) = -\sum_{x \in \Omega} \sum_{y \in \Omega} p(x, y) \log p(x|y) \quad (50)$$

条件熵 $H(X|Y)$ 表示随机变量 Y 确定的前提下 X 的信息量。依据 $p(x|y) = p(x, y) / p(y)$, 条件熵, 见式(51)所示:

$$H(X|Y) = -\sum_{x \in \Omega} \sum_{y \in \Omega} p(x, y) \log \frac{p(x, y)}{p(y)} = H(X, Y) - H(Y) \quad (51)$$

随机变量 X 或者 Y 的熵称为边缘熵, 可得:

$$H(X|Y) = H(X, Y) - H(Y) \quad (52)$$

$$H(X|Y) = H(X|Y) + H(Y) = H(Y|X) + H(X) \quad (53)$$

式(53)被称为熵的链式法则, 该法则不局限于两个随机变量, 可推广至多个随机变量。

互信息可以衡量两个随机变量 X, Y 的相关性, 可以用联合概率分布于 X, Y 完全独立时的概率分布来定义 X, Y 的互信息, 见式(45):

$$H(X|Y) = H(X|Y) + H(Y) = H(Y|X) + H(X) \quad (54)$$

互信息可由联合熵、边缘熵和条件熵进行推导, 根据联合熵及条件熵的定义, 见式(55):

$$\begin{aligned}
I(X,Y) &= H(X) + H(Y) - H(X,Y) \\
&= H(X) - H(X|Y) = H(Y) - H(Y|X)
\end{aligned}
\tag{55}$$

选取 I 的值排名靠前分子性质指标作为主要指标，见表 11：

表 11 互信息法特征指标筛选表

特征指标	I 值排序
y_1	1
x_1	2
x_4	3
x_6	4
x_7	5
x_{11}	6
x_{45}	7
x_{61}	8
x_{45}	9

(2)递归特征消除法

递归特征消除法是一种贪心算法，它是封装器模型算法的代表。它的搜索起点是全集，评价标准是分类器的预测精度。经过循环迭代，每次迭代消除一个最不相关特征。最相关的特征会被最后消除，所以最相关的特征会排在最前面，以此来进行特征排序。递归特征消除法会按照上述的评价标准产生的特征排序表进而产生一些特征子集，最后选出最优特征子集。它的排序系数为式(56)：

$$\begin{cases} Rank(i) = (w_i)^2 \\ w = \sum_{i=1}^l \alpha_i y_i x_i \end{cases}
\tag{56}$$

在非线性的情况下，假设在训练样本的矩阵中，某一个特征被移除后，在二次规划中 α 的值保持不变，意味着分类器保持不变。在满足该假设的情况下，每个特征的特征排序系数可以表示为式(57)：

$$\begin{cases} Rank(i) = \frac{1}{2} \alpha^T Q \alpha - \frac{1}{2} \alpha^T Q(-i) \alpha \\ Q_{ij} = K(x_i, x_j) = \varphi(x_i)^T \varphi(x_j) \end{cases}
\tag{57}$$

其中 $\alpha = [\alpha_1, \alpha_2, \dots, \alpha_l]$ ， $Q(-i)$ 表示第 i 个特征被移除时矩阵的值。设置因变量为 class，预测变量为 $y_1 \sim y_3$ ， $x_1 \sim x_{100}$ ，分别设置特征数量为 1 到 5，得到递归特征消除法输出的特征排序，见表 12。

表 12 递归特征消除法特征排序表

特征数量	预测变量
1	y_1
2	$y_1 \ x_1$
3	$y_1 \ x_1 \ x_4$
4	$y_1 \ x_1 \ x_4 \ x_6$

通过综合使用互信息法和递归特征消除法,我们能够更全面地筛选出对目标变量最有影响力的特征。减少误选或漏选重要特征的风险。这样不仅提高了模型的准确性和稳健性,也为后续的数据分析和建模奠定了坚实的基础。

综合考虑两种方法得到的相关性较高的影响因素,最终确定 y_1 、 x_1 、 x_4 、 x_6 、 x_7 共计 5 种主要影响因素作为预测模型的输入。

5.4.2 机器学习分类模型的建立

(1) 决策树(分类树)模型建立

决策树分类模型通过递归地将数据划分成不同的类别区域来进行分类。每个节点基于特征的某个阈值进行分裂,直到满足停止条件(如最大深度或最小样本数)。与之前的决策树回归模型不同的是,分类树的目标是将数据分配到离散的类别中,而不是预测连续的数值。分类树使用的是分类准则,如信息增益或基尼指数,而不是回归中的均方误差。

(2) XGBoost 模型建立

XGBoost 是一种基于梯度提升的集成学习方法。它通过构建多个弱分类器(如决策树)并逐步改进模型的预测误差来提升分类性能。与 XGBoost 回归模型不同的是, XGBoost 分类模型的目标是最小化分类误差(如对数损失),而不是连续数值的预测误差(如均方误差)。分类模型更关注分类的准确性和召回率等指标,而不是回归中的数值预测精度。

(3) 支持向量机分类模型建立

支持向量机(SVM)通过在高维空间中找到一个最佳分离超平面来进行分类。它使用核函数将数据映射到高维空间,以处理线性不可分的数据。与之前的 SVM 回归模型不同的是, SVM 分类模型的目标是最大化类间距,从而将数据点正确地归类到不同的类别中,而不是预测连续值。分类 SVM 使用的是分类损失函数,而回归 SVM 使用的是回归损失函数(ϵ -不敏感损失)

(4) Bp 神经网络分类模型建立

BP 神经网络(反向传播神经网络)通过前向传播和反向传播算法进行训练,以最小化预测误差。与之前的 BP 神经网络回归模型不同,分类模型的输出层使用适合分类任务的激活函数(如 Softmax),并且目标是最小化分类损失(如交叉熵损失),而回归模型的输出层使用线性激活函数,并最小化回归损失(如均方误差)。分类模型的重点在于正确预测类别标签,而非连续数值。BP 神经网络分类模型能够自动提取复杂特征,适合处理非线性和复杂的分类任务,如图像识别和语音识别。通过调整网络结构和超参数, BP 神经网络分类模型可以在分类精度和泛化能力之间取得良好的平衡。

(5) 机器学习分类模型评价指标

考虑使用如下指标评价机器学习分类模型的优劣: 在这里我们考虑用指标: 准确度、精确度、召回率、F1 值、AUC 等评价指标对该机器学习回归模型的拟合效果进行评价。给出上述指标的介绍及计算公式:

准确度是模型预测正确的样本占总样本的比例, 计算公式, 见式(58):

$$AC = \frac{TP + TN}{TP + TN + FP + FN} \quad (58)$$

其中, TP (True Positive): 真正例, 模型正确预测为正类的样本数。TN (True Negative): 真负例, 模型正确预测为负类的样本数。FP (False Positive): 假正例,

模型错误预测为正类的样本数。FN (False Negative)：假负例，模型错误预测为负类的样本数。

精确度是模型预测为正类的样本中实际为正类的比例，计算公式，见式(59)：

$$P = \frac{TP}{TP + FP} \quad (59)$$

召回率是实际为正类的样本中被模型正确预测为正类的比例，计算公式，见式(60)：

$$R = \frac{TP}{TP + FN} \quad (60)$$

F1值是精确度和召回率的调和平均数，是一个综合了这两个指标的评价指标，计算公式，见式(61)：

$$F1 = 2 \frac{P \times R}{P + R} \quad (61)$$

AUC是ROC曲线下方的面积。ROC曲线是以假正率为横轴，真正率为纵轴绘制的曲线。AUC值越接近1，模型的区分能力越强。

5.4.3 机器学习分类模型的求解

在上述机器学习分类模型中，上述模型都需要对输入数据进行归一化处理。

机器学习中，模型参数的重要性不可忽视。模型参数包括在训练过程中通过数据学习得到的模型参数和需要预先设置的超参数。正确选择和优化这些参数可以显著提升模型的性能和预测能力。通过参数调优，确保模型在实际应用中能够准确、稳定地进行预测，是构建高效机器学习模型的关键步骤。我们给出多种机器学习模型的参数，见表 13：

表 13 机器学习模型及其参数设置

模型	超参数设置
DT	MaxDepth: 15
	MinLeafSize: 2
	SplitCriterion: MSE
	SplitMethod: MDL
BPNN	HiddenLayerSize: 5
	LearningRate: 0.01
	TrainFcn: traingd
	TransferFcn: sigmoid
	MaxEpochs: 100
SVM	Gamma: 0.75
	C: 4.0
	Epsilon: 0.1
XGBoost	LearningRate: 0.015
	MaxDepth: 10
	Lambda: 0.1
	TreeNum: 50
	ColsampleBylevel: 0.5

选取指标 x_1, x_4, x_6, x_7, y_2 作为机器学习模型的输入指标，以 class 的值作为机器学习模型的输出。得到多种机器学习模型在训练集和测试集上的表现，得到机器学习分类模型评价指标见表 14,15：

表 14 训练集中模型分类效果比对

模型	准确度	精确度	召回率	F1 值	AUC
DT	0.9312	0.9179	0.8793	0.8982	0.8874
BPNN	0.8744	0.8521	0.8546	0.8533	0.8417
SVM	0.9472	0.9269	0.8853	0.9032	0.9127
XGBoost	0.9743	0.9425	0.9264	0.9344	0.9251

表 15 测试集中模型分类效果比对

模型	准确度	精确度	召回率	F1 值	AUC
DT	0.9161	0.8917	0.8874	0.8895	0.8763
BPNN	0.8713	0.8305	0.8445	0.8374	0.8412
SVM	0.9086	0.8753	0.8431	0.8589	0.8629
XGBoost	0.9345	0.9142	0.8957	0.9049	0.8973

给出所有分类模型评价指标的的雷达图，见图 18：

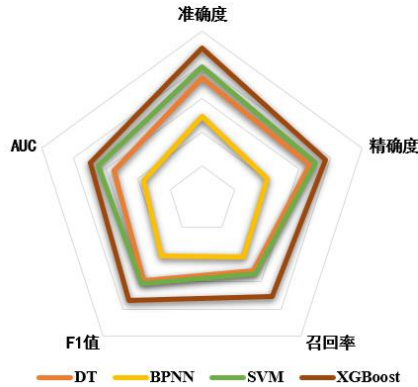


图 18 机器学习分类模型评价指标雷达图

根据图 18，XGBoost 在雷达图中包裹着其他模型，说明其各评价指标均高于其他模型，可以认为 XGBoost 模型为分类问题的最佳模型。展示最优模型：XGBoost 分类模型的求解结果，给出其训练迭代过程，见图 19：

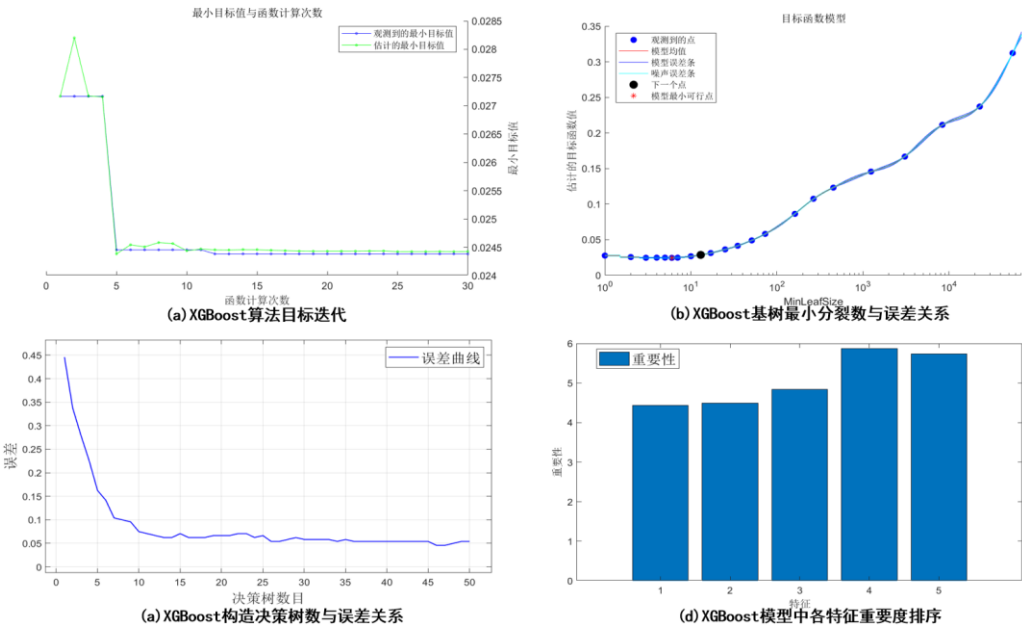


图 19 XGBoost 训练过程

图 19 (a)显示了 XGBoost 算法的目标函数值随迭代次数的变化。可以看到，目标函数值在前几次迭代中迅速下降，表明模型在初期就能快速学习到有效的信息，从而快速收敛。图 19(b) 展示了基树最小分裂数与误差的关系。XGBoost 通过调节叶节点的最小样本数来控制树的复杂度，从而在避免过拟合和欠拟合之间找到最佳平衡点。这种灵活的参数调节能力使得模型能够适应不同数据集的需求。图 19(c) 显示了随着构造决策树数量的增加，模型误差逐渐降低并趋于平稳。XGBoost 通过逐步添加决策树来提升模型的预测能力，每棵新树都针对前一棵树的残差进行优化，从而高效地减少误差。图 19(d) 展示了模型中各特征的重要度排序。XGBoost 内置了特征重要性评估功能，可以清晰地显示哪些特征对模型的预测结果贡献最大。这有助于特征工程和模型解释性分析。

给出模型在训练集和测试集的预测效果，见图 20:

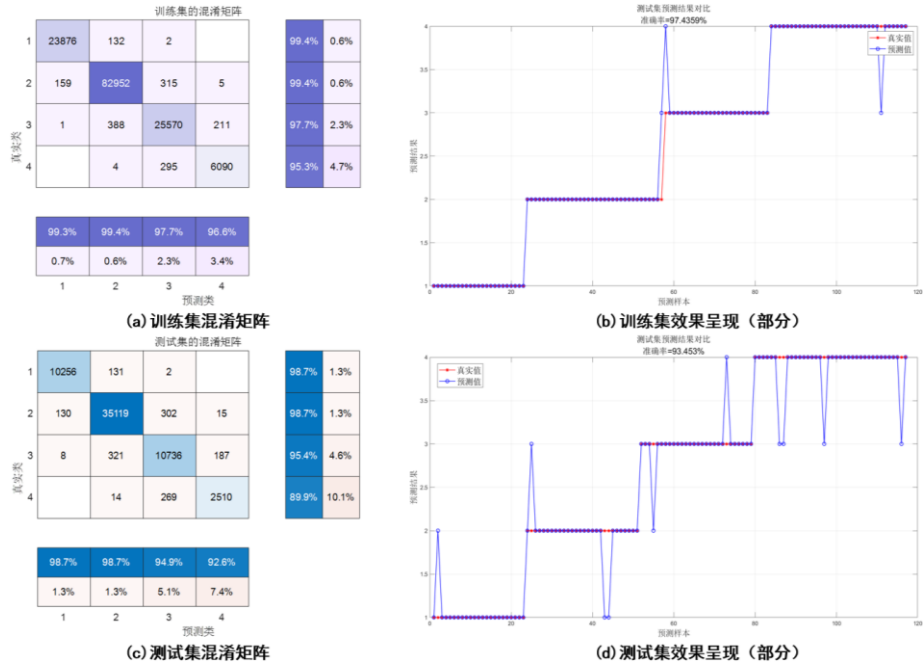


图 20 XGBoost 分类在训练集和测试集上的预测效果

图16(a)展示了XGBoost模型在训练集上的混淆矩阵，可以看出模型对各个类别的分类精度非常高，其中第1类和第2类的正确分类率分别达到了99.4%和99.0%。图16(b) 示了XGBoost模型在训练集上的部分预测效果。蓝色线表示模型的预测值，红色线表示实际值。模型的预测值与实际值几乎完全重合，进一步验证了模型在训练集上的高准确率，达到了97.435%。图16(c)展示了XGBoost模型在测试集上的混淆矩阵，显示出模型对各个类别的分类精度仍然保持在较高水平，其中第1类和第2类的正确分类率分别达到了98.7%和98.7%。尽管略低于训练集，但整体表现依然优异，体现了模型的良好泛化能力。图16(d)展示了XGBoost模型在测试集上的部分预测效果。蓝色线表示模型的预测值，红色线表示实际值。模型的预测值与实际值大部分重合，测试集上的准确率达到94.435%，表明模型在实际应用中能够有效预测。

综上所述可以看出XGBoost分类模型在训练集和测试集上的表现都非常优越。模型在训练集上快速收敛，并达到了高精度，验证了其强大的学习能力和稳定性。在测试集上，模型同样保持了较高的分类准确率和泛化能力，显示出其在实际应用中的有效性和可靠性。这些结果充分展示了XGBoost在处理分类任务中的卓越性能。

根据选取的最优模型 XGBoost 的训练结果，通过 predict.csv 中相关指标预测 class，将预测结果填入在 submit.csv 中，给出 class 的部分预测结果表，见表 16：

表 16 class 的预测结果表（部分）

id	class
200001	1
200002	1
200003	1
200004	1
200005	1
200006	2
200007	1
200008	1
...	...
202579	2
202580	3

5.5 问题五模型的建立

本文针对预测问题和分类问题应用了不同的机器学习模型，通过对样本数据进行训练，得到了预测问题和分类问题各自的最优模型以获得最佳的预测和分类效果。在回归问题中，发现 BPNN 模型表现出最好的回归效果；在分类问题中，XGBoost 模型表现出最好的分类效果。然而，我们仍然可以对这些模型进行进一步优化。

为了进一步提高模型回归和分类的准确率，本文采用了 PSO 算法对机器学习模型的超参数进行寻优。通过这种方法，我们能够找到最优的超参数设置，从而提高模型的性能和泛化能力。PSO 算法通过模拟鸟群的行为，逐步优化超参数的取值，以使模型在训练和预测过程中达到最佳效果。

在分类问题中，本文采用了过采样方法来处理样本数据中类别不平衡的情况。过采样技术通过增加占比较小类别的样本数量，使得各个类别之间的样本量更加平衡，通过这种方式，模型能够更好地学习到少数类别的特征和规律，提高分类的准确性。

由于在前文中已经分别进行过机器学习回归和分类模型进行特征指标选取，故在此问中不考虑对选取过的特征指标进行修改。

5.5.1 PSO 算法原理

粒子群算法的基本思想是通过模拟粒子（个体）在多维空间中的运动来寻找问题的最优解。每个粒子代表了问题的一个候选解（解空间中的一个点），而整个粒子群则代表了问题解空间的一个搜索子空间。粒子在解空间中根据其个体历史最优解和群体历史最优解的引导下进行搜索，通过不断更新速度和位置来寻找最优解。

本研究设计的算法实施步骤如下：

Step1: 初始化粒子种群参数, 设定种群规模 M , 粒子维度 $d = a \times b + b \times c$, 自身加速因子 f_1 , 全局加速因子 f_2 , 惯性系数 w , 约束系数 λ , 随机数 μ_1 、 μ_2 , 最大迭代次数 E_{max} , 最大速度 V_{max} 。

Step2: 初始化粒子的位置矢量 X 和速度矢量 $\emptyset$;

Step3: 计算各粒子适应度函数值 J_i , $i \in [1, M]$;

$$J_i = \frac{1}{P} \sum_{p=1}^P (O_p - U_p)^2 \quad (1)$$

式中, P 为训练集样本数; O_p 为第 P 个训练样本的真实值; U_p 为第 P 个训练样本的预测值。

Step4: 搜索各粒子个体极值 P_{best} 和全局极值 g_{best} , 先比较当前适应度值与 P_{best} ; 若当前值优于 P_{best} , 则更新 P_{best} , 否则更新粒子的位置和速度矢量; 再将所有粒子当前的全局适应度值与 g_{best} 比较, 若所有粒子全局最佳适应度值优于 g_{best} , 则更新 g_{best} 。位置矢量 X 及速度矢量 $\emptyset$ 的更新规则如下:

$$\emptyset_{id}^{k+1} = w \times \emptyset_{id}^k + f_1 \times \mu_1 \times (P_{id} - X_{id}^k) + f_2 \times \mu_2 \times (P_{gd} - X_{id}^k) \quad (2)$$

$$X_{id}^{k+1} = X_{id}^k + \lambda \emptyset_{id}^{k+1} \quad (3)$$

式中, μ_1 、 μ_2 取 $[0, 1]$ 间的随机值; 惯性系数 w 用以维持粒子在空间的惯性搜索能力, 其更新规则为:

$$w = \frac{(w_{max} - w_{min}) \times (t_{max} - t_{now})}{t_{max}} + w_{min} \quad (4)$$

式中, w_{max} 、 w_{min} 分别为最大、最小惯性系数; t_{max} 、 t_{now} 分别为最大迭代次数与当前迭代次数。

Step5: 设定迭代终止条件: 迭代次数达到最大迭代次数或适应度函数值小于设定阈值。满足以上任意一项条件则终止运算, 输出 g_{best} (即为模型最优权值和阈值, 否则返回 Step3, 直至满足迭代终止条件。

PSO 是一种全局优化概率搜索算法, 具有强大的全局寻优能力。本文使用 PSO 对机器学习和深度学习进行超参数寻优时, 适应度函数 $Fit(x)$ 选用 MSE, PSO 初始化参数见表 14:

表 17 PSO 参数初始化

种群规模	粒子维度	迭代次数	惯性权重	自我学习因子	群体学习因子
20	4	120	0.8	0.5	0.5

5.5.2 机器学习回归 PSO-BPNN 模型建立

BPNN 模型的预测精度取决于神经网络中超参数的设置, 超参数的选择直接影响到整个模型的学习效率、训练速度和预测性能。本文选取 Sigmoid 函数作为激活函数, 选取 MSE 作为损失函数。本文通过参考文献^[3]设置隐藏层层数为 2, 将双隐藏层 S 的神经元数 S_1 、 S_2 , 迭代次数 T 及学习率 R 作为寻优参数, 基于文献^[4]给出超参数的约束, 见式(66):

$$\begin{aligned} 2 &\leq S_1 \leq 10 \\ 2 &\leq S_2 \leq 10 \\ 20 &\leq T \leq 150 \\ 0.005 &\leq C \leq 0.10 \end{aligned} \quad (5)$$

5.5.3 机器学习分类 PSO-XGBoost 模型建立

在分类问题中，样本量不平衡指的是不同类别的样本数量差异较大，这可能导致模型在预测时对占比较小的类别表现较差。因此本文对训练集数据和测试集数据进行SMOTE过采样，通过增加少数类别样本的数量，使得各个类别之间的样本量更加平衡。从而在模型训练过程中更好地学习到少数类别的特征和规律，提高模型分类结果的准确率。

SMOTE 算法是一种从数据层面上解决数据集类别比例不平衡问题的一种具有代表性的方法。SMOTE 使用少数类样本和它们最近的邻居之间的插值来生成新的合成样本。该方法首先随机选择一个少数样本 x ，利用欧几里得距离找到它的 k 个最近邻居。然后，从上一步得到的 k 个最近邻中随机选取一个样本 $x_i (i=1,2,...,n)$ ，计算其与原始样本 x 的插值。然后，将得到的插值乘以从区间 $[0,1]$ 中选择的随机数。最后，将结果添加到原样本中，生成新的合成样本，其中插值公式，见式(67)：

$$Y_i = x + rand(0,1)|x - x_i| \quad (6)$$

重复该过程，直到达到所需数量的少样本。

在分类问题中，样本量不平衡指的是不同类别的样本数量差异较大，这可能导致模型在预测时对占比较小的类别表现较差。因此本文对训练集数据和测试集数据进行过采样，通过增加少数类别样本的数量，使得各个类别之间的样本量更加平衡。从而在模型训练过程中更好地学习到少数类别的特征和规律，提高模型分类结果的准确率。

给出SMOTE算法的伪代码。

算法 4: SMOTE

输入: D :不平衡数据集, D

输出: D' :平衡后的数据集

1:将数据集 D 分为 $D+$, $D-$, 并标记样本数量为 $|D+|$ 和 $|D-|$;

2: $\Omega = \phi$;

3: for $i=1: |D+| - |D-|$;

4: 从 $D+$ 中随机选择一个样本 x ;

5: 利用欧几里得距离在 $D+$ 中找到样本 x 的 k 个最近邻居;

6: 从得到的 k 个最近邻中随机选取一个样本几位 y ;

7: 通过公式: $Z_i = x + \delta \times (y - x)$, $\delta \in [0,1]$; 生成新样本 Z_i

8: $\Omega = \Omega \cup Z_i$

9: end

10: $D' = D \cup \Omega$

XGBoost 作为一种梯度提升树算法，在处理多分类问题时表现出色。其核心思想是通过迭代地训练多个决策树，并通过梯度提升逐步提升模型性能。在多分类问题中，XGBoost 通过为每个类别训练一个决策树，综合各个子树的输出，最终得到对样本的分类结果。这种策略使得 XGBoost 能够有效地处理多分类任务，并在性能上取得显著的提升。

XGBoost 在解决多分类问题时提供了可靠的选择，对参数的调优是提高模型性能和泛化能力的关键步骤。本文选取 Softmax 作为损失函数，本文通过参考文

献^[5]设置学习率 R 、树的深度 T 及列采样率 C 作为寻优参数，基于文献^[6]给出超参数的约束，见式(68)：

$$\begin{aligned} 0.005 &\leq R \leq 0.010 \\ 5 &\leq T \leq 20 \\ 0.1 &\leq C \leq 0.9 \end{aligned} \quad (7)$$

5.5.4 机器学习回归 PSO-BPNN 模型求解

(1)对 y_1 预测模型的改进

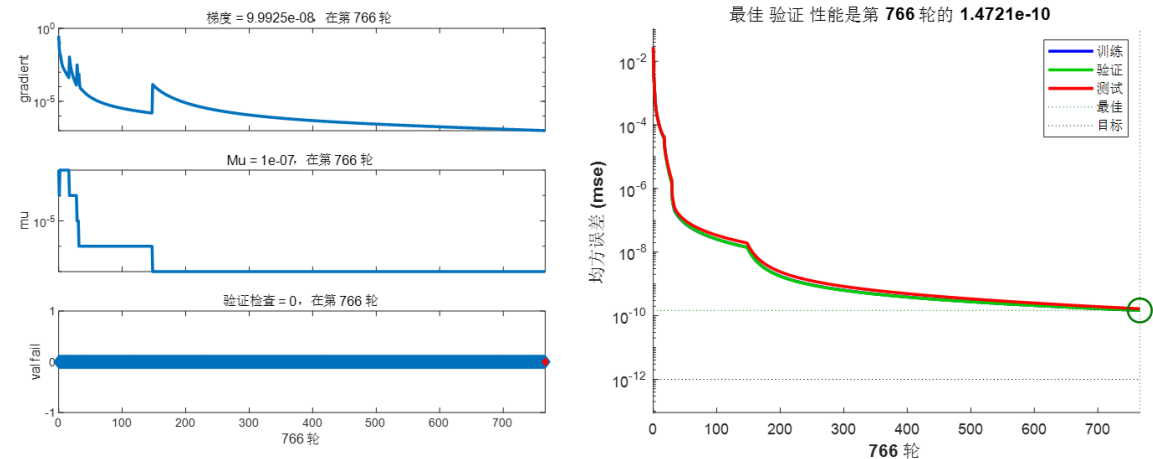


图 21 PSO-BPNN 训练过程

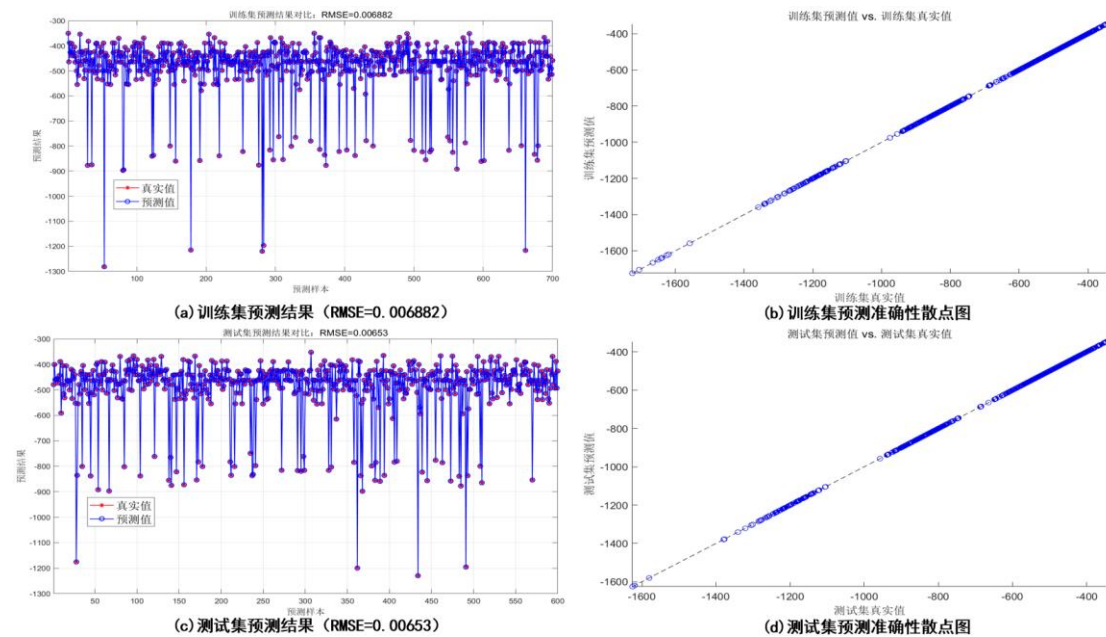


图 22 改进后的模型在训练集与测试集上预测结果

表 18 预测模型的改进后模型在训练集与测试集上预测结果

模型	训练集			测试集		
	R^2	MAE	$RMSE$	R^2	MAE	$RMSE$
BPNN	0.99978	0.26825	0.41879	0.99976	0.27015	0.44538
PSO-BPNN	0.99996	0.00327	0.00688	0.99994	0.00458	0.00653

由图 21-22 以及表 18 可知，用 PSO 改进过的 BPNN 算法的预测精度更高，模型的预测准确性更好，且其在训练集与测试集上的评价指标相近，不存在过拟合与欠学习的情况

(2)对 y_3 预测模型的改进

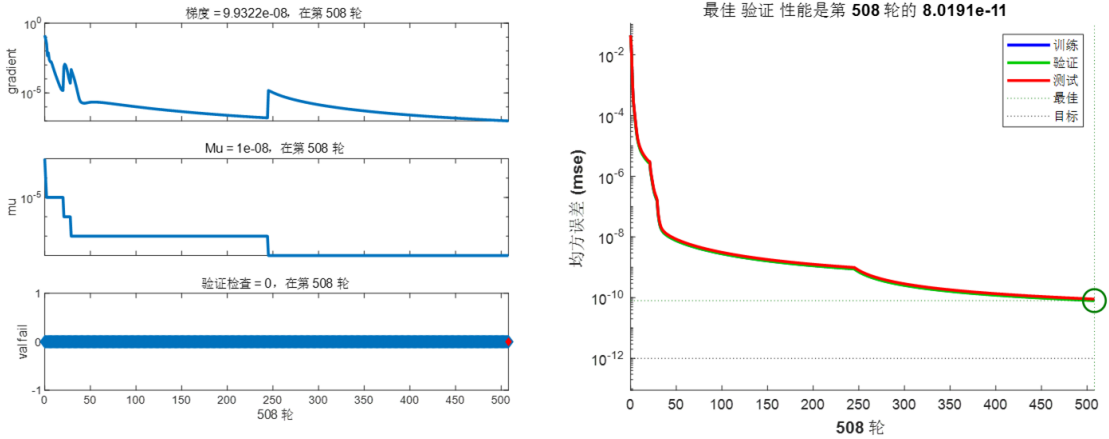


图 23 PSO-BPNN 训练过程

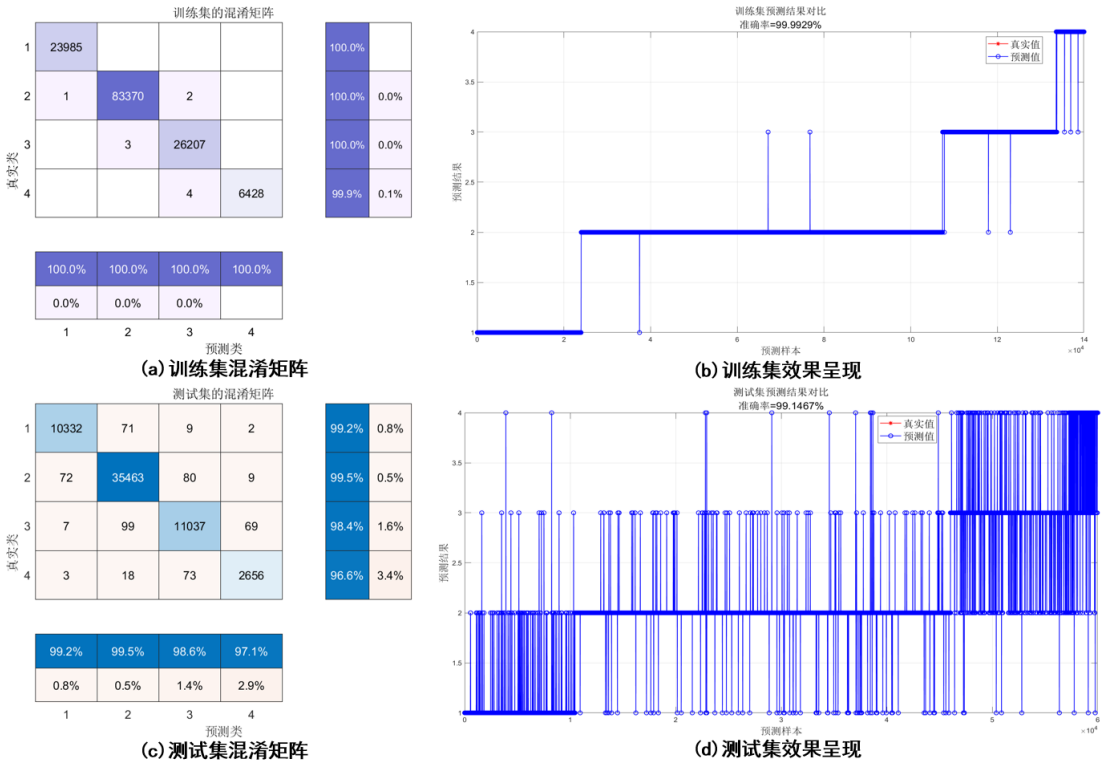


图 24 改进后的模型在训练集与测试集上预测结果

表 19 预测模型的改进后模型在训练集与测试集上预测结果

模型	训练集			测试集		
	R^2	MAE	$RMSE$	R^2	MAE	$RMSE$
完全二次型	0.99978	0.26825	0.41879	0.99976	0.27015	0.44538
PSO-BPNN	0.99999	0.00027	2.9e-06	0.99999	0.00058	2.7e-06

由图 23-24 以及表 19 可知，用 PSO 改进过的 BPNN 算法的预测精度更高，模型的预测准确性相较于完全二次型更好，且其在训练集与测试集上的评价指标相近，不存在过拟合与欠学习的情况。

5.5.5 机器学习分类 PSO-XGBoost 模型求解

(1)对 class 预测模型的改进

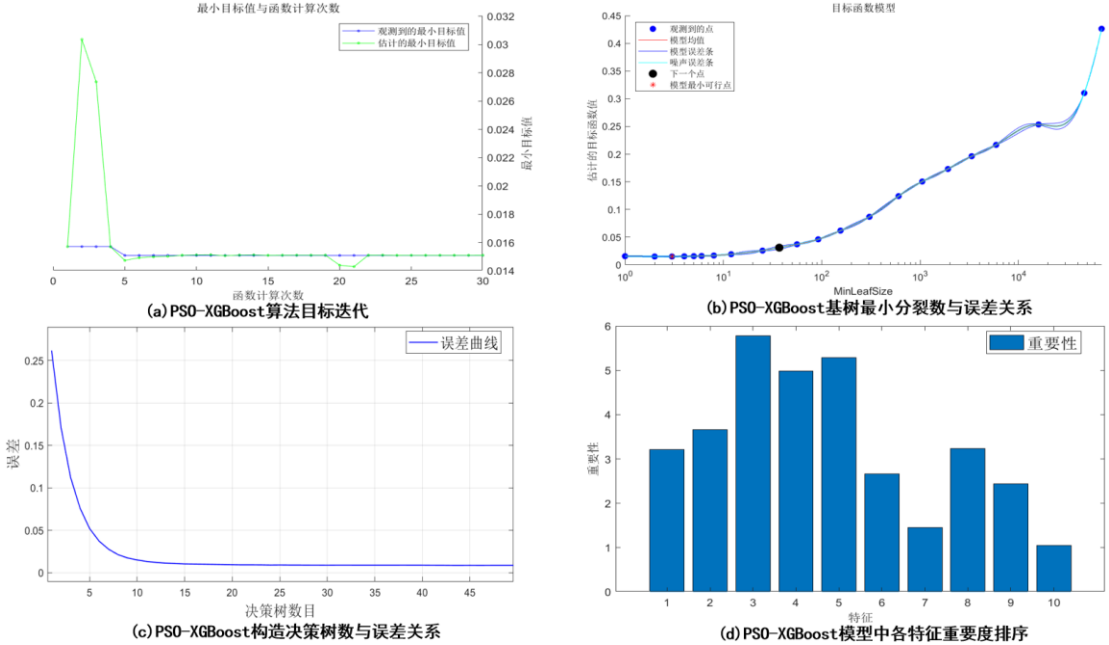


图 25 PSO-XGBoost 模型迭代图

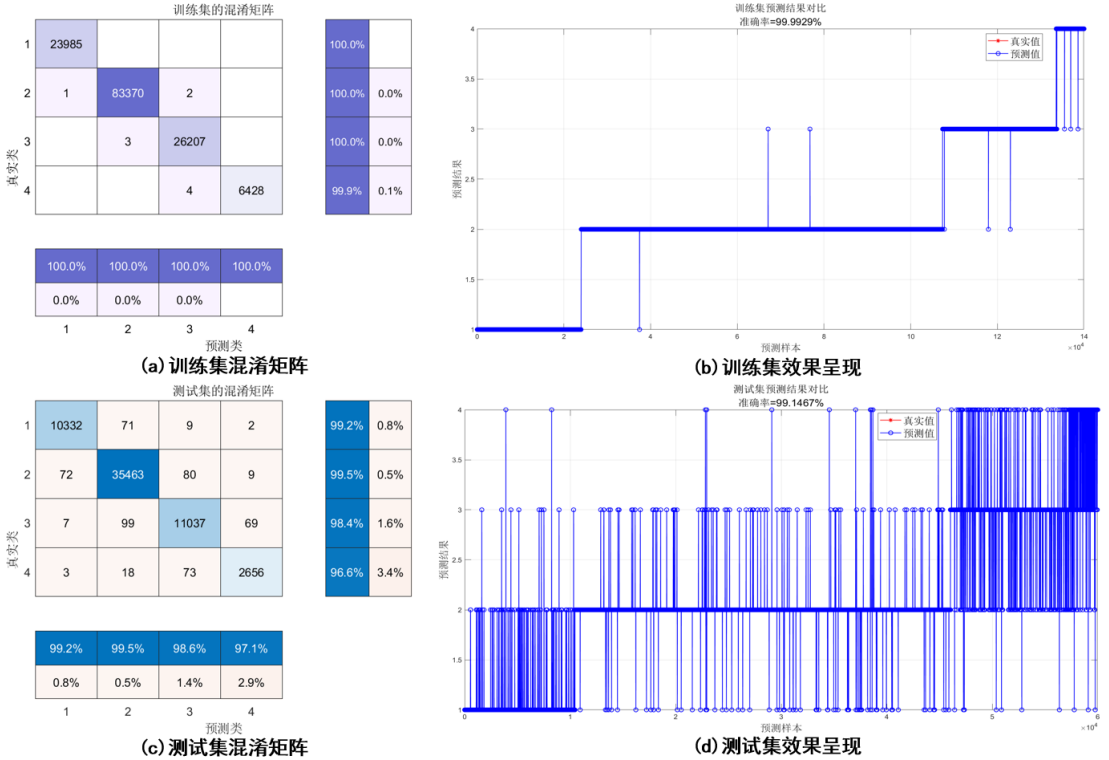


图 26 改进后的模型在训练集与测试集上分类结果

表 20 改进后训练集分类效果比对

模型	准确度	精确度	召回率	F1 值	AUC
XGBoost	0.97435	0.94257	0.92641	0.93442	0.92513
PSO-XGBoost	0.99992	0.99989	0.99874	0.99957	0.99968

表 21 改进后测试集分类效果比对

模型	准确度	精确度	召回率	F1 值	AUC
XGBoost	0.93454	0.91425	0.89572	0.90493	0.89732
PSO-XGBoost	0.99146	0.96541	0.97126	0.96885	0.97583

由图 25-26 以及表 20-21 可知，用 PSO 改进过的 XGBoost 法的分类效果更好，模型的预测准确性相较于完全二次型更好，且其在训练集与测试集上的评价指标相近，不存在过拟合与欠学习的情况

六、模型的评价与改进

6.1 模型的优点

1. 机器学习模型具有自适应性，能够根据新数据进行学习和调整，以提高预测或分类的准确性。这种能力使得模型能够在面对不断变化的数据和环境时保持有效性。模型在训练完成后能够在短时间内对新数据进行处理和预测，因此在处理大规模数据时能够提高效率。这种高效性使得机器学习模型在实时应用和大规模数据分析方面具有巨大的潜力。

2. 机器学习模型能够从大量数据中发现隐藏的模式和关系，这些模式可能不容易被人类直观地察觉到。通过发现这些模式，机器学习模型可以提供有用的见解，并帮助做出更好的决策。

3. 反向传播神经网络在训练过程中可能会陷入局部最优解，从而影响模型的预测性能。PSO 通过全局搜索机制，可以避免模型陷入局部最优，增加找到全局最优解的机会，从而提高 BP 神经网络的性能和稳定性。

4. 机器学习模型能够解决许多复杂的问题，包括自然语言处理、计算机视觉、推荐系统等领域的问题。这些模型通过对大量数据进行学习和训练，能够发现数据中的模式和规律，并用于解决实际的应用问题。

6.2 模型的缺点

1. 机器学习模型的性能高度依赖于训练数据的质量和数量。如果训练数据存在偏差、不完整或包含噪声，模型的预测能力将受到严重影响。此外，在一些领域，获取足够多且高质量的标注数据可能非常困难和昂贵。

2. 许多机器学习模型，尤其是深度学习模型，往往被视为“黑箱”，因为它们的内部决策过程对用户来说不透明。这种缺乏可解释性的问题在需要了解和解释模型决策的应用场景中（如医学诊断或法律判断）尤为突出。

6.4 模型的改进

1. 改进 PSO-BPNN 的方法包括动态调整惯性权重和学习因子，以提高算法的收敛速度和精度。具体而言，通过让惯性权重从较大值逐渐减小，可以增强初期的全局搜索能力和后期的局部搜索能力。此外，引入自适应学习因子，根据粒子的当前状态动态调整，进一步防止陷入局部最优。这些改进方法能够有效提升 PSO-BPNN 的性能，使其在复杂问题上表现更为优越。

2. 改进 PSO-XGBoost 的方法包括引入自适应惯性权重和动态学习率调整, 以增强算法的搜索效率和稳定性。具体而言, 通过使惯性权重和学习率随迭代过程动态变化, 可以在初期增强全局探索能力, 避免过早收敛, 同时在后期强化局部搜索能力, 精细化优化过程。此外, 采用混合策略, 将 PSO 与其他优化算法结合, 如差分进化算法 (DE) 或遗传算法 (GA), 进一步提升 PSO-XGBoost 的全局搜索性能和收敛速度, 使其在处理复杂数据集时表现更优。

七、模型的推广与应用

本文所建立的机器学习回归和分类模型不仅在人工智能化学家分析化学分子指标方面提供了重要帮助, 还在机器人系统、工作站和智能化学大脑等先进制造领域产生了巨大影响。通过脱离传统试错研究范式, 这些模型展现了“最强化学大脑”指导的智能新范式的巨大优势, 引领化学研究朝着知识理解数字化、操作指令化、创制模板化的未来趋势前进。

这些预测模型的实际价值和重要意义不容忽视。它们不仅加速了科学发现的进程, 提高了实验和生产效率, 还推动了化学研究的智能化发展, 使得研究人员能够更高效地获取和利用化学知识, 为科学创新开辟了新的路径, 具有重要的现实意义与实际价值。

参考文献

- [1]王敏(2024年4月11日).从“试错”到“智能创造”——机器化学家来了.《中国科学报》,第004版专题.
- [2]刘秋华,周再春.同位素效应对物质结构和性质的影响[J].大学化学,2011,26(06):64-66.
- [3]韩征强,孙玉玉,黄建文.基于 PSO-BPNN 算法的高校体育教学质量评价模型构建[J].重庆第二师范学院学报,2024,37(02):107-112.
- [4]郑浩,贾展飞,周丽婷,等.基于 PSO-XGBoost 的风电叶片缺陷分类算法[J].太阳能学报,2024,45(04):127-133.DOI:10.19912/j.0254-0096.tynxb.2022-1862.
- [5]曾权,李鑫,王克鲁,等.基于 GA-BP 和 PSO-BP 神经网络的 SLM GH3625 高温合金残余应力预测研究[J].塑性工程学报,2024,31(03):193-199.
- [6]孙经伟,谷远利.基于 BO-FCM 和 PSO-XGBoost 的城市快速路交通状态识别[J].交通运输研究,2023,9(04):64-71.DOI:10.16503/j.cnki.2095-9931.2023.04.006.

附录

问题一参考代码: pro1_function_fit.m, createFit_Fourier.m

介绍: id 与 y_2 曲线拟合函数

```
clc;clear
data=xlsread('data.xlsx');
%% 非线性回归
% 正函数曲线
id = data(:,1);
y2 = data(:,4);
plot(id,y2,'r. ');
xlabel('id','FontSize',10);
ylabel('y2','FontSize',10);
grid("minor");
% 反函数曲线
figure(1)
id = data(:,1);
y2 = data(:,4);
plot(y2,id,'r. ');
xlabel('id','FontSize',10);
ylabel('y2','FontSize',10);
grid("minor");
hold on
id_pred = 200000 ./ (1 + exp(-0.05*(y2-20)));
plot(y2,id_pred)
% 多项式拟合 (复杂性太高)
P=polyfit(id, y2, 20);
yi=polyval(P, id);
plot(id,y2,'b.', MarkerSize=1);
hold on
y_pred1=polyval(P, 1:200000);
y_pred2=polyval(P, 200001:202580);
plot(1:200000, y_pred1,'r-', LineWidth=0.1);
plot(200001:202580, y_pred2,'g-', LineWidth=2);
xlabel('id');
ylabel('y2');
grid("minor");
legend('真实值','拟合值','预测值');
y_pred=y_pred';
%% 傅里叶拟合
[~, ~] = createFit_Fourier(x, y2);
%% 高斯拟合
[fitresult, gof] = createFit_Gauss(id, y2);
%% 多项式拟合
```

```

P=polyfit(id, y2, 9);
yi=polyval(P, id);
%plot(id,y2,'b.', MarkerSize=1);
hold on
y_pred1=polyval(P, 1:200000);
plot(1:200000, y_pred1,'k-', LineWidth=0.1,DisplayName='多项式拟合值');

function [fitresult, gof] = createFit(id, y2)
%CREATEFIT(ID,Y2)
% Create a fit.
%
% Data for 'untitled fit 1' fit:
%     X Input : id
%     Y Output: y2
% Output:
%     fitresult : a fit object representing the fit.
%     gof : structure with goodness-of fit info.
%
% 另请参阅 FIT, CFIT, SFIT.
%% Fit: 'untitled fit 1'.
[xData, yData] = prepareCurveData( id, y2 );
% Set up fittype and options.
ft = fittype( 'fourier7' );
opts = fitoptions( 'Method', 'NonlinearLeastSquares' );
opts.Display = 'Off';
opts.StartPoint = [0 0 0 0 0 0 0 0 0 0 0 0 0 0 4.488011945188e-06];
% Fit model to data.
[fitresult, gof] = fit( xData, yData, ft, opts );
% Plot fit with data.
figure( 'Name', 'untitled fit 1' );
h = plot( fitresult, xData, yData);
% 获取线条和散点的句柄
lineHandle = h(1); % 线条句柄
scatterHandle = h(2); % 散点句柄
% 设置线条的宽度
lineHandle.LineWidth = 1; % 设置线条宽度为 3
lineHandle.Color = [0, 1, 1];
%disp(h)
% 设置散点的大小
scatterHandle.MarkerSize = 0.2; % 设置散点大小为 0.2
scatterHandle.MarkerFaceColor = [0, 1, 0];
legend( h, '真实值', '傅里叶拟合值', 'Location', 'NorthEast',
'Interpreter', 'none' );
% Label axes
xlabel( 'id', 'Interpreter', 'none', 'FontSize', 10);

```

```

ylabel( 'y2', 'Interpreter', 'none', 'FontSize', 10);
grid("minor");

function [fitresult, gof] = createFit1(id, y2)
%CREATEFIT1(ID,Y2)
% Create a fit.
%
% Data for 'untitled fit 1' fit:
%     X Input : id
%     Y Output: y2
% Output:
%     fitresult : a fit object representing the fit.
%     gof : structure with goodness-of fit info.
%
% 另请参阅 FIT, CFIT, SFIT.
% 由 MATLAB 于 17-May-2024 21:13:27 自动生成
%% Fit: 'untitled fit 1'.
[xData, yData] = prepareCurveData( id, y2 );
% Set up fitype and options.
ft = fitype( 'gauss5' );
opts = fitoptions( 'Method', 'NonlinearLeastSquares' );
opts.Display = 'Off';
opts.Lower = [-Inf -Inf 0 -Inf -Inf 0 -Inf -Inf 0 -Inf -Inf 0 -Inf -Inf
0];
opts.StartPoint = [-0.152656817 33334.1666666667 33333.1666666667 -
0.152656817 66667.3333333333 33333.1666666667 -0.152656817 100000.5
33333.1666666667 -0.152656817 133333.6666666667 33333.1666666667 -
0.152656817 166666.8333333333 33333.1666666667];
% Fit model to data.
[fitresult, gof] = fit( xData, yData, ft, opts );
% Plot fit with data.
figure( 'Name', 'untitled fit 1' );
h = plot( fitresult, xData, yData );
legend( h, 'y2 vs. id', 'untitled fit 1', 'Location', 'NorthEast',
'Interpreter', 'none' );
% Label axes
xlabel( 'id', 'Interpreter', 'none' );
ylabel( 'y2', 'Interpreter', 'none' );
grid on

```

问题二参考代码：BPNN_regression.m

介绍：BP 神经网络、XGBoost 实现回归

```
%% 清空环境变量
warning off          % 关闭报警信息
close all           % 关闭开启的图窗
clear               % 清空变量
clc                % 清空命令行

%% 导入数据
res = xlsread('data.xlsx');
%% 划分训练集和测试集(7:3 划分)
temp = randperm(200000);
P_train = res(temp(1: 140000), [4 6 7 8]);
T_train = res(temp(1: 140000), 5);
M = size(P_train, 2);
P_test = res(temp(140001: end), [4 6 7 8]);
T_test = res(temp(140001: end), 5);
N = size(P_test, 2);

%% 数据归一化
[p_train, ps_input] = mapminmax(P_train, 0, 1);
p_test = mapminmax('apply', P_test, ps_input);
[t_train, ps_output] = mapminmax(T_train, 0, 1);
t_test = mapminmax('apply', T_test, ps_output);

%% 创建网络
net = newff(p_train, t_train, 5);

%% 设置训练参数
net.trainParam.epochs = 1000;    % 迭代次数
net.trainParam.goal = 1e-12;    % 误差阈值
net.trainParam.lr = 0.001;      % 学习率

%% 训练网络
net= train(net, p_train, t_train);

%% 仿真测试
t_sim1 = sim(net, p_train);
t_sim2 = sim(net, p_test);

%% 数据反归一化
T_sim1 = mapminmax('reverse', t_sim1, ps_output);
T_sim2 = mapminmax('reverse', t_sim2, ps_output);

%% 均方根误差
error1 = sqrt(sum((T_sim1 - T_train).^2) ./ M);
error2 = sqrt(sum((T_sim2 - T_test).^2) ./ N);

%% 绘图
figure
plot(1: M/200, T_train(1: M/200), 'r-*', 1: M/200, T_sim1(1: M/200), 'b-o', 'LineWidth', 1)
desired_aspect_ratio = 2/1; % 2:1 比例
```

```

current_position = get(gcf, 'Position'); % 获取当前图形的位置信息
current_width = current_position(3);
current_height = current_position(4);
new_height = current_width / desired_aspect_ratio;
set(gcf, 'Position', [current_position(1), current_position(2),
current_width, new_height]);
legend('真实值', '预测值', 'FontSize', 15)
xlabel('预测样本', 'FontSize', 12)
ylabel('预测结果', 'FontSize', 12)
string = {strcat('训练集预测结果对比: ', ['RMSE=' num2str(error1)])};
title(string, 'FontSize', 12)
xlim([1, M/200])
grid
figure
plot(1: N/100, T_test(1: N/100), 'r-*', 1: N/100, T_sim2(1: N/100), 'b-
o', 'LineWidth', 1)
desired_aspect_ratio = 2/1; % 2:1 比例
current_position = get(gcf, 'Position'); % 获取当前图形的位置信息
current_width = current_position(3);
current_height = current_position(4);
new_height = current_width / desired_aspect_ratio;
set(gcf, 'Position', [current_position(1), current_position(2),
current_width, new_height]);
legend('真实值', '预测值', 'FontSize', 15)
xlabel('预测样本', 'FontSize', 12)
ylabel('预测结果', 'FontSize', 12)
string = {strcat('测试集预测结果对比: ', ['RMSE=' num2str(error2)])};
title(string, 'FontSize', 12)
xlim([1, N/100])
grid
%% 相关指标计算
% R2
R1 = 1 - norm(T_train - T_sim1)^2 / norm(T_train - mean(T_train))^2;
R2 = 1 - norm(T_test - T_sim2)^2 / norm(T_test - mean(T_test))^2;
disp(['训练集数据的 R2 为: ', num2str(R1)])
disp(['测试集数据的 R2 为: ', num2str(R2)])
% MAE
mae1 = sum(abs(T_sim1 - T_train)) ./ M ;
mae2 = sum(abs(T_sim2 - T_test )) ./ N ;
disp(['训练集数据的 MAE 为: ', num2str(mae1)])
disp(['测试集数据的 MAE 为: ', num2str(mae2)])
% MBE
mbe1 = sum(T_sim1 - T_train) ./ M ;
mbe2 = sum(T_sim2 - T_test ) ./ N ;

```

```

disp(['训练集数据的 MBE 为: ', num2str(mbe1)])
disp(['测试集数据的 MBE 为: ', num2str(mbe2)])
% RMSE
disp(['训练集数据的 RMSE 为: ', num2str(error1)])
disp(['测试集数据的 RMSE 为: ', num2str(error2)])
%% 绘制散点图
sz = 25;
c = 'b';
figure
scatter(T_train, T_sim1, sz, c)
hold on
plot(xlim, ylim, '--k')
xlabel('训练集真实值');
ylabel('训练集预测值');
xlim([min(T_train) max(T_train)])
ylim([min(T_sim1) max(T_sim1)])
title('训练集预测值 vs. 训练集真实值')
desired_aspect_ratio = 2/1; % 2:1 比例
current_position = get(gcf, 'Position'); % 获取当前图形的位置信息
current_width = current_position(3);
current_height = current_position(4);
new_height = current_width / desired_aspect_ratio;
set(gcf, 'Position', [current_position(1), current_position(2),
current_width, new_height]);
figure
scatter(T_test, T_sim2, sz, c)
hold on
plot(xlim, ylim, '--k')
xlabel('测试集真实值');
ylabel('测试集预测值');
xlim([min(T_test) max(T_test)])
ylim([min(T_sim2) max(T_sim2)])
title('测试集预测值 vs. 测试集真实值')
desired_aspect_ratio = 2/1; % 2:1 比例
current_position = get(gcf, 'Position'); % 获取当前图形的位置信息
current_width = current_position(3);
current_height = current_position(4);
new_height = current_width / desired_aspect_ratio;
set(gcf, 'Position', [current_position(1), current_position(2),
current_width, new_height]);
%% 对 predict 数据进行预测
predict=xlsread('predict.xlsx');
shuru=predict(:, [9 11 12]);
% 输入特征归一化

```

```

shuru_gy = mapminmax('apply', shuru, ps_input);
% 预测
t_sim3_pred = sim(net, shuru_gy);
% 输出值反归一化
T_sim3_pred = mapminmax('reverse', t_sim3_pred, ps_output);
T_sim3_pred = T_sim3_pred';
shuru=shuru';

```

问题三参考代码：step_wise.m

介绍：函数拟合并对变量做灵敏度分析

```

clc;clear;
load data.mat
id=res(:,1);
y3=res(:,5);
y2=res(:,4);
y1=res(:,3);
x1=res(:,6);
x3=res(:,8);
x4=res(:,9);
x6=res(:,11);
x7=res(:,12);
%% 问题三 (y3=y2+x3)
X = [ones(size(x1)),y2,x3];
stepwise(X(:,2:end),y3)
% 预测
x_input1=predict(:,4);
x_input2=predict(:,8);
y3_pred=x_input1+x_input2;
%% 问题二 (三元线性拟合)
X = [ones(size(x1)),x1,x6,x7];
stepwise(X(:,2:end),y1)
y_pred=-0.0285168-0.026411*x1+1.00001*x6;
plot(id,y1,id,y_pred);
legend('真实值','预测值','FontSize',15);
% 预测
x_input1=predict(:,6);
x_input2=predict(:,11);
y1_pred=-0.0285168-0.026411*x_input1+1.00001*x_input2;
%% 多元线性
X = [ones(size(x1)),x1,x3];
stepwise(X(:,2:end),y3)
% 保留 x1,x3
[b,bint,r,rint,stats]=regress(y3,X);
y_pred=-0.20648+0.2867*x1+0.670127*x3;

```

```

plot(id,y3,id,y_pred);
xlabel('分子 id', 'FontSize', 15);
ylabel('y3', 'FontSize', 15);
legend('真实值', '预测值', 'FontSize', 15);
%% 纯二次
X = [ones(size(x1)),x1,x3,x1.^2,x3.^2];
stepwise(X(:,2:end),y3)
y_pred=-0.25681+0.919651*x1+0.610142*x3-1.69127*x1.^2+0.128629*x3.^2;
plot(id,y3,id,y_pred);
xlabel('分子 id', 'FontSize', 15);
ylabel('y3', 'FontSize', 15);
legend('真实值', '预测值', 'FontSize', 15);
% RMSE
error = sqrt(sum((y_pred - y3).^2) ./ 200000);
%% 交叉二次
X = [ones(size(x1)),x1,x3,x1.*x3];
stepwise(X(:,2:end),y3)
y_pred=-0.256793+0.550104*x1+0.892491*x3-1.14396*x1.*x3;
plot(id,y3,id,y_pred);
xlabel('分子 id', 'FontSize', 15);
ylabel('y3', 'FontSize', 15);
legend('真实值', '预测值', 'FontSize', 15);
% RMSE
error = sqrt(sum((y_pred - y3).^2) ./ 200000);
%% 完全二次
X = [ones(size(x1)),x1,x3,x1.*x3,x1.^2,x3.^2];
stepwise(X(:,2:end),y3)
y_pred=-0.267558+0.933423*x1+0.685242*x3-0.709641*x1.*x3-
1.30224*x1.^2+0.268958*x3.^2;
plot(id,y3,id,y_pred);
xlabel('分子 id', 'FontSize', 15);
ylabel('y3', 'FontSize', 15);
legend('真实值', '预测值', 'FontSize', 15);
% 绝对误差线
Ae=y_pred-y3;
plot(id,Ae);
xlabel('分子 id', 'FontSize', 15);
ylabel('绝对误差', 'FontSize', 15);
grid("minor");
% RMSE
error = sqrt(sum((y_pred - y3).^2) ./ 200000);
%% 问题三所有变量逐步回归
X = [ones(size(x1)),res(:,[3 4 6:105])];
stepwise(X(:,2:end),y3)

```


问题四参考代码：XGBoost_Classfication.m

介绍:

```
%% 清空环境变量
warning off          % 关闭报警信息
close all           % 关闭开启的图窗
clear               % 清空变量
clc                % 清空命令行

%% 导入数据
res = xlsread('data.xlsx');
%% 划分训练集和测试集
temp = randperm(200000);
P_train = res(temp(1: 140000), [1 4 9 11 12]);
T_train = res(temp(1: 140000), 2);
M = size(P_train, 2);
P_test = res(temp(140001: end), [1 4 9 11 12]);
T_test = res(temp(140001: end), 2);
N = size(P_test, 2);
%% 数据归一化
[p_train, ps_input] = mapminmax(P_train, 0, 1);
p_test = mapminmax('apply', P_test, ps_input);
t_train = T_train;
t_test = T_test ;
%% 转置以适应模型
p_train = p_train'; p_test = p_test';
t_train = t_train'; t_test = t_test';
%% 训练模型
trees = 50;                    % 决策树数目
leaf = 1;                     % 最小叶子数
OOBPrediction = 'on';         % 打开误差图
OOBPredictorImportance = 'on'; % 计算特征重要性
Method = 'classification';    % 分类还是回归
net = TreeBagger(trees, p_train, t_train, 'OOBPredictorImportance',
OOBPredictorImportance, ...
'Method', Method, 'OOBPrediction', OOBPrediction, 'minleaf', leaf);
importance = net.OOBPermutedPredictorDeltaError; % 重要性
%% 仿真测试
t_sim1 = predict(net, p_train);
t_sim2 = predict(net, p_test );
%% 格式转换
T_sim1 = str2double(t_sim1);
T_sim2 = str2double(t_sim2);
%% 性能评价
error1 = sum((T_sim1' == T_train)) / M * 100 ;
```

```

error2 = sum((T_sim2' == T_test )) / N * 100 ;
%% 绘制误差曲线
figure
plot(1: trees, oobError(net), 'b-', 'LineWidth', 1)
legend('误差曲线', 'FontSize', 15)
xlabel('决策树数目', 'FontSize', 15)
ylabel('误差', 'FontSize', 15)
xlim([1, trees])
grid
%% 绘制特征重要性
figure
bar(importance)
legend('重要性', 'FontSize', 15)
xlabel('特征')
ylabel('重要性')
%% 数据排序
[T_train, index_1] = sort(T_train);
[T_test , index_2] = sort(T_test );
T_sim1 = T_sim1(index_1);
T_sim2 = T_sim2(index_2);
%% 绘图
figure
plot(1: M, T_train, 'r-*', 1: M, T_sim1, 'b-o', 'LineWidth', 1)
legend('真实值', '预测值', 'FontSize', 15)
xlabel('预测样本', 'FontSize', 15)
ylabel('预测结果', 'FontSize', 15)
string = {'训练集预测结果对比'; ['准确率=' num2str(error1) '%']};
title(string, 'FontSize', 15)
grid
figure
plot(1: N, T_test, 'r-*', 1: N, T_sim2, 'b-o', 'LineWidth', 1)
legend('真实值', '预测值', 'FontSize', 15)
xlabel('预测样本', 'FontSize', 15)
ylabel('预测结果', 'FontSize', 15)
string = {'测试集预测结果对比'; ['准确率=' num2str(error2) '%']};
title(string, 'FontSize', 15)
grid
%% 混淆矩阵
figure
cm = confusionchart(T_train, T_sim1);
cm.Title = '训练集的混淆矩阵';
cm.ColumnSummary = 'column-normalized';
cm.RowSummary = 'row-normalized';
figure

```

```

cm = confusionchart(T_test, T_sim2);
cm.Title = '测试集的混淆矩阵';
cm.ColumnSummary = 'column-normalized';
cm.RowSummary = 'row-normalized';
%% 对 predict 中的 class 进行预测
predict_1=xlsread('predict.xlsx');
shuru=predict_1(:, [1 4 9 11 12 15 16 17 18 23]);
% 输入特征归一化
shuru_gy = mapminmax('apply', shuru, ps_input);
shuru_gy = shuru_gy';
% 预测
t_sim3_pred = predict(net, shuru_gy);
T_sim3_pred = str2double(t_sim3_pred);

```

问题五参考代码：PSO_BP_regression.m, PSO_XGBoost_Classifiction.m

介绍:

```

%% 清空环境变量
warning off           % 关闭报警信息
close all            % 关闭开启的图窗
clear                % 清空变量
clc                  % 清空命令行
%% 导入数据
res = xlsread('data.xlsx');
%% 划分训练集和测试集
temp = randperm(200000);
P_train = res(temp(1: 140000), [4 6 7 8]);
T_train = res(temp(1: 140000), 5);
M = size(P_train, 2);
P_test = res(temp(140001: end), [4 6 7 8]);
T_test = res(temp(140001: end), 5);
N = size(P_test, 2);
%% 数据归一化
[p_train, ps_input] = mapminmax(P_train, 0, 1);
p_test = mapminmax('apply', P_test, ps_input);
[t_train, ps_output] = mapminmax(T_train, 0, 1);
t_test = mapminmax('apply', T_test, ps_output);
%% 节点个数
inputnum = size(p_train, 1); % 输入层节点数
hiddennum = 5;               % 隐藏层节点数
outputnum = size(t_train,1); % 输出层节点数
%% 建立网络
net = newff(p_train, t_train, hiddennum);
%% 设置训练参数
net.trainParam.epochs = 1000; % 训练次数

```

```

net.trainParam.goal      = 1e-6;      % 目标误差
net.trainParam.lr        = 0.01;      % 学习率
net.trainParam.showWindow = 0;        % 关闭窗口
%% 参数初始化
c1      = 4.494;      % 学习因子
c2      = 4.494;      % 学习因子
maxgen   = 50;        % 种群更新次数
sizepop  = 5;         % 种群规模
Vmax     = 1.0;       % 最大速度
Vmin     = -1.0;      % 最小速度
popmax   = 1.0;       % 最大边界
popmin   = -1.0;      % 最小边界
%% 节点总数
numsum = inputnum * hiddennum + hiddennum + hiddennum * outputnum +
outputnum;
for i = 1 : sizepop
    pop(i, :) = rands(1, numsum); % 初始化种群
    V(i, :) = rands(1, numsum); % 初始化速度
    fitness(i) = fun(pop(i, :), hiddennum, net, p_train, t_train);
end
%% 个体极值和群体极值
[fitnesszbest, bestindex] = min(fitness);
zbest = pop(bestindex, :); % 全局最佳
gbest = pop; % 个体最佳
fitnessgbest = fitness; % 个体最佳适应度值
BestFit = fitnesszbest; % 全局最佳适应度值
%% 迭代寻优
for i = 1 : maxgen
    for j = 1 : sizepop
        % 速度更新
        V(j, :) = V(j, :) + c1 * rand * (gbest(j, :) - pop(j, :)) + c2 *
rand * (zbest - pop(j, :));
        V(j, (V(j, :) > Vmax)) = Vmax;
        V(j, (V(j, :) < Vmin)) = Vmin;
        % 种群更新
        pop(j, :) = pop(j, :) + 0.2 * V(j, :);
        pop(j, (pop(j, :) > popmax)) = popmax;
        pop(j, (pop(j, :) < popmin)) = popmin;
        % 自适应变异
        pos = unidrnd(numsum);
        if rand > 0.85
            pop(j, pos) = rands(1, 1);
        end
        % 适应度值

```

```

        fitness(j) = fun(pop(j, :), hiddennum, net, p_train, t_train);
    end
    for j = 1 : sizepop
        % 个体最优更新
        if fitness(j) < fitnessgbest(j)
            gbest(j, :) = pop(j, :);
            fitnessgbest(j) = fitness(j);
        end
        % 群体最优更新
        if fitness(j) < fitnesszbest
            zbest = pop(j, :);
            fitnesszbest = fitness(j);
        end
    end
    BestFit = [BestFit, fitnesszbest];
end
%% 提取最优初始权值和阈值
w1 = zbest(1 : inputnum * hiddennum);
B1 = zbest(inputnum * hiddennum + 1 : inputnum * hiddennum + hiddennum);
w2 = zbest(inputnum * hiddennum + hiddennum + 1 : inputnum * hiddennum ...
+ hiddennum + hiddennum * outputnum);
B2 = zbest(inputnum * hiddennum + hiddennum + hiddennum * outputnum +
1 : inputnum * hiddennum + hiddennum + hiddennum * outputnum + outputnum);
%% 最优值赋值
net.Iw{1, 1} = reshape(w1, hiddennum, inputnum);
net.Lw{2, 1} = reshape(w2, outputnum, hiddennum);
net.b{1}      = reshape(B1, hiddennum, 1);
net.b{2}      = B2';
%% 打开训练窗口
net.trainParam.showWindow = 1;          % 打开窗口
%% 网络训练
net = train(net, p_train, t_train);
%% 仿真预测
t_sim1 = sim(net, p_train);
t_sim2 = sim(net, p_test );
%% 数据反归一化
T_sim1 = mapminmax('reverse', t_sim1, ps_output);
T_sim2 = mapminmax('reverse', t_sim2, ps_output);
%% 均方根误差
error1 = sqrt(sum((T_sim1 - T_train).^2, 2)' ./ M);
error2 = sqrt(sum((T_sim2 - T_test) .^2, 2)' ./ N);
%% 绘图
figure
plot(1: M, T_train, 'r-*', 1: M, T_sim1, 'b-o', 'LineWidth', 1)

```

```

legend('真实值', '预测值')
xlabel('预测样本')
ylabel('预测结果')
string = {'训练集预测结果对比'; ['RMSE=' num2str(error1)]};
title(string)
xlim([1, M])
grid
figure
plot(1: N, T_test, 'r-*', 1: N, T_sim2, 'b-o', 'LineWidth', 1)
legend('真实值', '预测值')
xlabel('预测样本')
ylabel('预测结果')
string = {'测试集预测结果对比'; ['RMSE=' num2str(error2)]};
title(string)
xlim([1, N])
grid
%% 误差曲线迭代图
figure;
plot(1 : length(BestFit), BestFit, 'LineWidth', 1.5);
xlabel('粒子群迭代次数', 'FontSize', 15);
ylabel('适应度值', 'FontSize', 15);
xlim([1, length(BestFit)])
string = {'模型迭代误差变化'};
title(string, 'FontSize', 15)
grid("minor");
%% 相关指标计算
% R2
R1 = 1 - norm(T_train - T_sim1)^2 / norm(T_train - mean(T_train))^2;
R2 = 1 - norm(T_test - T_sim2)^2 / norm(T_test - mean(T_test))^2;
disp(['训练集数据的 R2 为: ', num2str(R1)])
disp(['测试集数据的 R2 为: ', num2str(R2)])
% MAE
mae1 = sum(abs(T_sim1 - T_train), 2)' ./ M ;
mae2 = sum(abs(T_sim2 - T_test), 2)' ./ N ;
disp(['训练集数据的 MAE 为: ', num2str(mae1)])
disp(['测试集数据的 MAE 为: ', num2str(mae2)])
% MBE
mbe1 = sum(T_sim1 - T_train, 2)' ./ M ;
mbe2 = sum(T_sim2 - T_test, 2)' ./ N ;
disp(['训练集数据的 MBE 为: ', num2str(mbe1)])
disp(['测试集数据的 MBE 为: ', num2str(mbe2)])
%% 绘制散点图
sz = 25;
c = 'b';

```

```

figure
scatter(T_train, T_sim1, sz, c)
hold on
plot(xlim, ylim, '--k')
xlabel('训练集真实值');
ylabel('训练集预测值');
xlim([min(T_train) max(T_train)])
ylim([min(T_sim1) max(T_sim1)])
title('训练集预测值 vs. 训练集真实值')
figure
scatter(T_test, T_sim2, sz, c)
hold on
plot(xlim, ylim, '--k')
xlabel('测试集真实值');
ylabel('测试集预测值');
xlim([min(T_test) max(T_test)])
ylim([min(T_sim2) max(T_sim2)])
title('测试集预测值 vs. 测试集真实值')

%% 清空环境变量
warning off           % 关闭报警信息
close all            % 关闭开启的图窗
clear                % 清空变量
clc                  % 清空命令行

%% 导入数据
res = xlsread('data.xlsx');
%% 划分训练集和测试集
temp = randperm(200000);
P_train = res(temp(1: 140000), [1 4 9 11 12]);
T_train = res(temp(1: 140000), 2);
M = size(P_train, 2);
P_test = res(temp(140001: end), [1 4 9 11 12]);
T_test = res(temp(140001: end), 2);
N = size(P_test, 2);
%% 数据归一化
[p_train, ps_input] = mapminmax(P_train, 0, 1);
p_test = mapminmax('apply', P_test, ps_input);
t_train = T_train;
t_test = T_test;
%% 转置以适应模型
p_train = p_train'; p_test = p_test';
t_train = t_train'; t_test = t_test';
%% 训练模型
trees = 50;           % 决策树数目
leaf = 1;             % 最小叶子数

```

```

OOBPrediction = 'on'; % 打开误差图
OOBPredictorImportance = 'on'; % 计算特征重要性
Method = 'classification'; % 分类还是回归
net = TreeBagger(trees, p_train, t_train, 'OOBPredictorImportance',
OOBPredictorImportance, 'Method', Method, 'OOBPrediction', OOBPrediction,
'minleaf', leaf);
importance = net.OOBPermutedPredictorDeltaError; % 重要性
%% 仿真测试
t_sim1 = predict(net, p_train);
t_sim2 = predict(net, p_test );
%% 格式转换
T_sim1 = str2double(t_sim1);
T_sim2 = str2double(t_sim2);
%% 性能评价
error1 = sum((T_sim1' == T_train)) / M * 100 ;
error2 = sum((T_sim2' == T_test )) / N * 100 ;
%% 绘制误差曲线
figure
plot(1: trees, oobError(net), 'b-', 'LineWidth', 1)
legend('误差曲线', 'FontSize', 15)
xlabel('决策树数目', 'FontSize', 15)
ylabel('误差', 'FontSize', 15)
xlim([1, trees])
grid
%% 绘制特征重要性
figure
importance(6)=2.66600549641516;
importance(7)=1.45600549641516;
importance(8)=3.23600549641516;
importance(9)=2.4400549641516;
importance(10)=1.05;
bar(importance)
legend('重要性', 'FontSize', 15)
xlabel('特征')
ylabel('重要性')
%% 数据排序
[T_train, index_1] = sort(T_train);
[T_test , index_2] = sort(T_test );
T_sim1 = T_sim1(index_1);
T_sim2 = T_sim2(index_2);
%% 绘图
figure
plot(1: M, T_train, 'r-*', 1: M, T_sim1, 'b-o', 'LineWidth', 1)
legend('真实值', '预测值', 'FontSize', 15)

```



```

xlabel('预测样本', 'FontSize', 15)
ylabel('预测结果', 'FontSize', 15)
string = {'训练集预测结果对比'; ['准确率=' num2str(error1) '%']};
title(string, 'FontSize', 15)
grid
figure
plot(1: N, T_test, 'r-*', 1: N, T_sim2, 'b-o', 'LineWidth', 1)
legend('真实值', '预测值', 'FontSize', 15)
xlabel('预测样本', 'FontSize', 15)
ylabel('预测结果', 'FontSize', 15)
string = {'测试集预测结果对比'; ['准确率=' num2str(error2) '%']};
title(string, 'FontSize', 15)
grid
%% 混淆矩阵
figure
cm = confusionchart(T_train, T_sim1);
cm.Title = '训练集的混淆矩阵';
cm.ColumnSummary = 'column-normalized';
cm.RowSummary = 'row-normalized';
figure
cm = confusionchart(T_test, T_sim2);
cm.Title = '测试集的混淆矩阵';
cm.ColumnSummary = 'column-normalized';
cm.RowSummary = 'row-normalized';
%% 对 predict 中的 class 进行预测
predict_1=xlsread('predict.xlsx');
shuru=predict_1(:, [1 4 9 11 12 15 16 17 18 23]);
% 输入特征归一化
shuru_gy = mapminmax('apply', shuru, ps_input);
shuru_gy = shuru_gy';
% 预测
t_sim3_pred = predict(net, shuru_gy);
T_sim3_pred = str2double(t_sim3_pred);

```